Chapter 6

Path and Trajectory Planning

Pengcheng Hu, SMMG, HKUST(GZ)

pengchenghu@hkust-gz.edu.cn

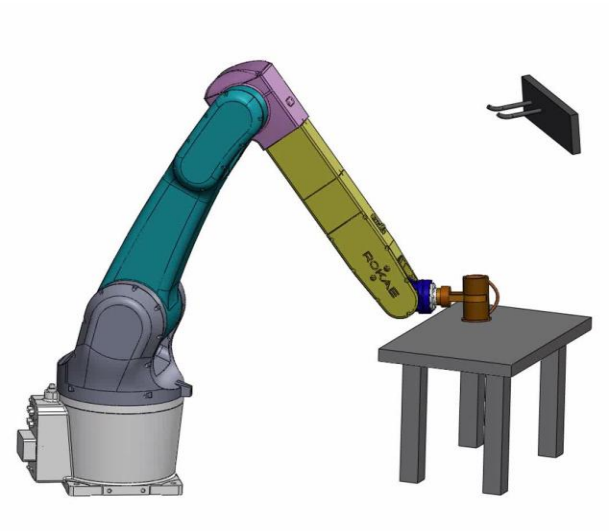
Chapter 6 Path and Trajectory Planning

1. Introduction
2. Configuration space
3. Path planning for $Q = R^2$
4. Artificial potential field
5. Sampling-based methods
6. Trajectory planning

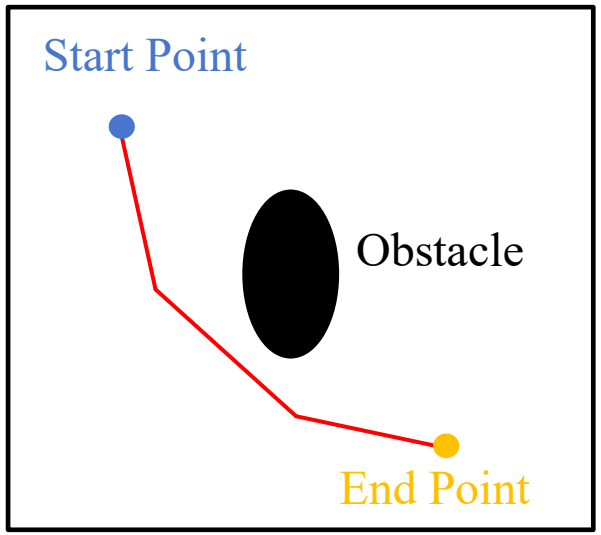
1. Introduction

☐ Path and trajectory planning

- Path planning provides a geometric description of robot motion, **does NOT** consider any dynamic aspects of the motion.
- Trajectory planning computes a function $q(t)$ that completely specifies the **desired motion** of the robot as it traverses the path.

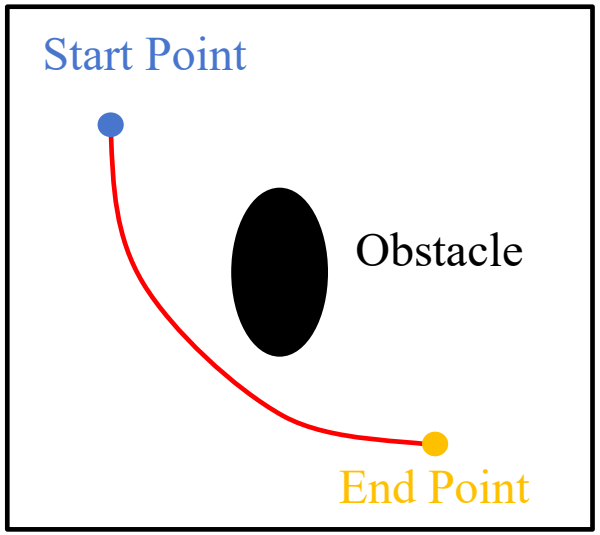


Cup-picking and placing problem



Path planning

Interpolate the desired path and generate **timebased** control set points



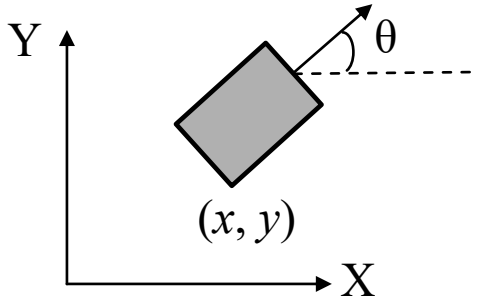
Trajectory generation $q(t)$

2. Configuration space

Representing the Configuration Space ($\mathcal{Q}$)

- Robot's configuration q: a complete specification of the location of every point on the robot
- Configuration space: set of all possible configurations

A rigid object moving in the plane:

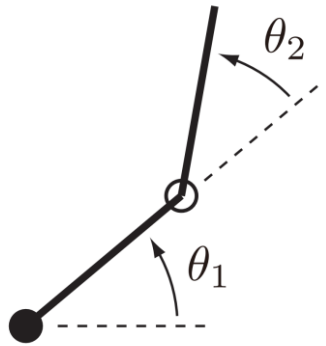


$$q = (x, y, \theta)$$

$$\mathcal{Q} = \mathbb{R}^2 \times SO(1)$$

Position on the plane — Rotation

Two-link planar arm:



$$q = (\theta_1, \theta_2)$$

$$\mathcal{Q} = S^1 \times S^1$$

S^1 represents the orientation of the object as a point on the unit circle

2. Configuration space

Configuration Space Obstacles ($\mathcal{QO}$)

- A collision occurs when any point on the robot contacts either an object in the workspace or the boundary of the workspace (e.g., walls or the floor) - purposes of planning collision free paths
- **Configuration space obstacle**: the set of configurations for which the robot collides with an obstacle

The subset of the workspace that is occupied by the *i*-th obstacle

Configuration space obstacle: $\mathcal{QO} = \{q \in \mathcal{Q} | \mathcal{A}(q) \cap \mathcal{O} \neq \emptyset\}$ $\mathcal{O} = \cup \mathcal{O}_i$

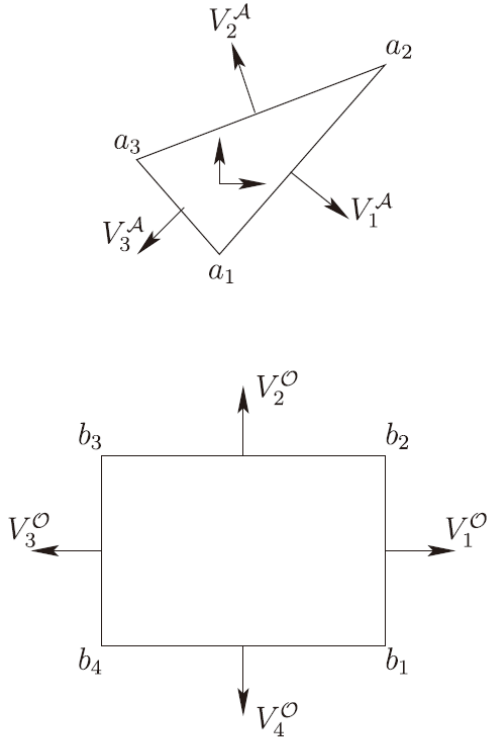
The subset of the workspace that is occupied by the robot at configuration q

The set of **collision-free configurations**: $\mathcal{Q}_{free} = \mathcal{Q} \backslash \mathcal{QO}$

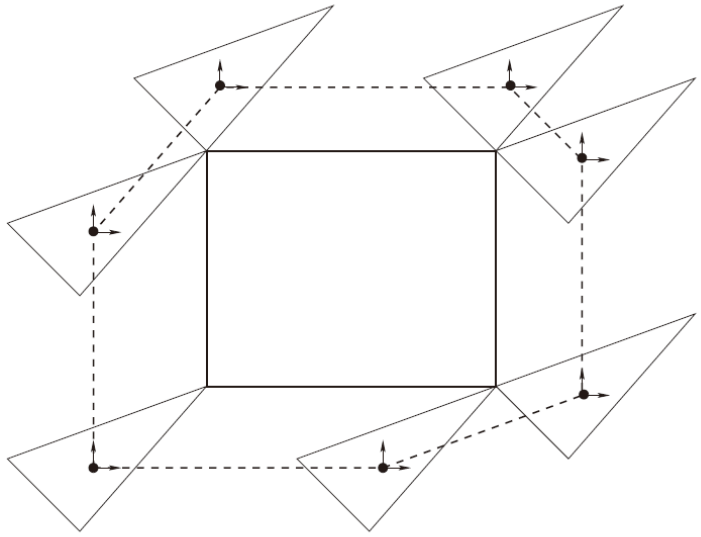
2. Configuration space

Configuration Space Obstacles

- **Example**: the robot is a triangle-shaped rigid object in a two-dimensional workspace that contains a single rectangular obstacle.



Robot and obstacle



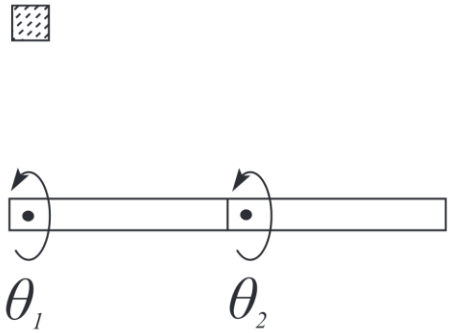
Configuration space obstacle

The boundary of the **configuration space obstacle** $\boldsymbol{QO}$ (shown as a dashed line) can be obtained by computing the convex hull of the configuration at which the robot makes vertex-to-vertex contact with the single convex obstacle.

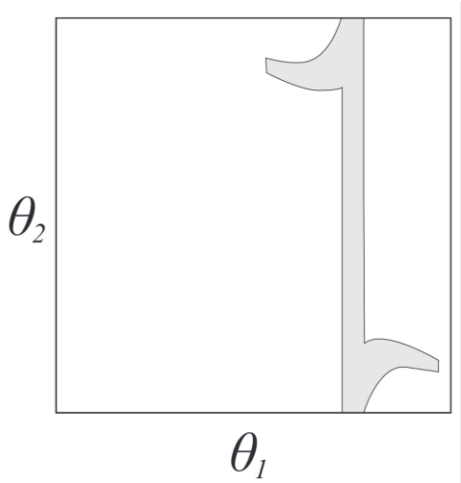
2. Configuration space

Configuration Space Obstacles

- **Example**: A 2R planar arm
  - Configuration: $q = (\theta_1, \theta_2)$.
  - The region $\mathcal{QO}$ was computed using a **discrete grid** on the configuration space. For each cell in the grid, a collision test was performed, and the cell was shaded when a collision occurred.



A two-link planar arm contains a small polygonal obstacle

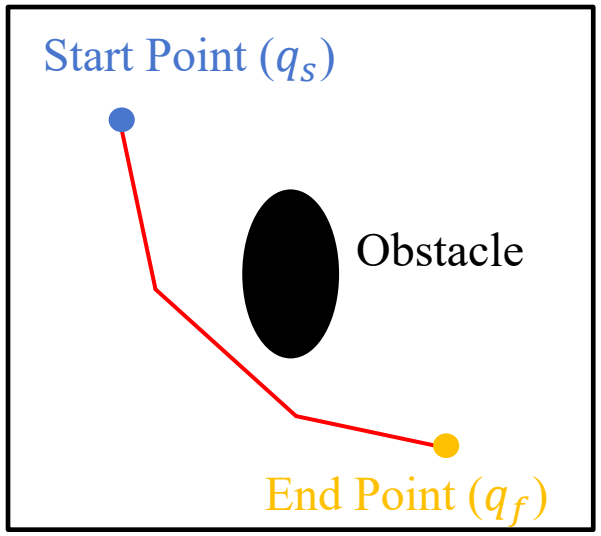


Configuration space obstacle

2. Configuration space

☐ Paths in the Configuration Space

- **Path planning problem**: find a path from an initial configuration q_s to a final configuration q_f, such that the robot does not collide with any obstacle as it traverses the path.



3. Path Planning for $Q = R^2$

Problem definition for $Q = R^2$

- Robot and obstacles can be represented as polygons in the plane, and robot can move in any arbitrary direction
- The robot's workspace (also its configuration space) is bounded, e.g., mobile robots navigate in workspaces such as warehouses, factory floors, office buildings, etc.

Three graph-based algorithms that can be used to construct collision free paths

- (1) The Visibility Graph
- (2) The Generalized Voronoi Diagram
- (3) Trapezoidal Decompositions

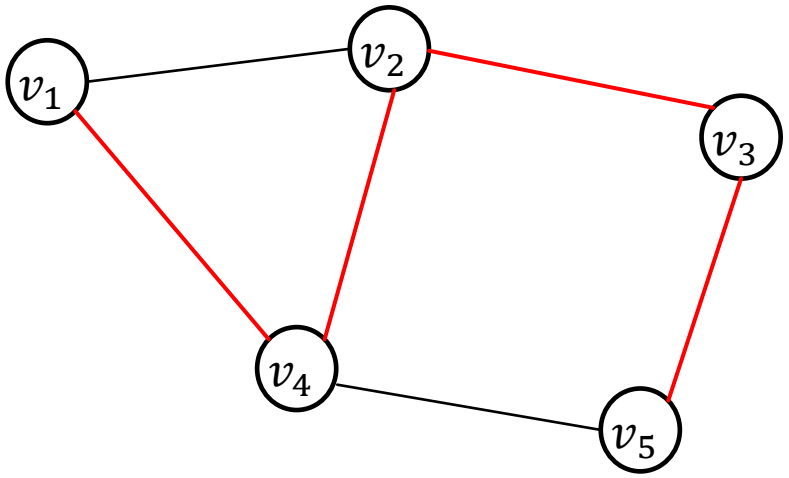
(1) Graph-based method

- Build graph data structures to represent the set of possible paths
- The path planning problem is reduced to a graph search problem

3. Path Planning for $Q = R^2$

Basics of Graph

- A graph is defined as $G = (V, E)$, in which V is a set of vertices, and E is a set of edges, each of which corresponds to a pair of vertices.
- The edges $(v_1, v_4), (v_4, v_2), (v_2, v_3), (v_3, v_5)$ define a path from v_1 to v_5.



Graph

$$G = (V, E)$$

$$V = \{v_1, v_2, \ldots, v_5\}$$

$$E = \{(v_1, v_2),\ (v_2, v_3), (v_2, v_4), (v_1, v_4), (v_4, v_5), (v_3, v_5)\}$$

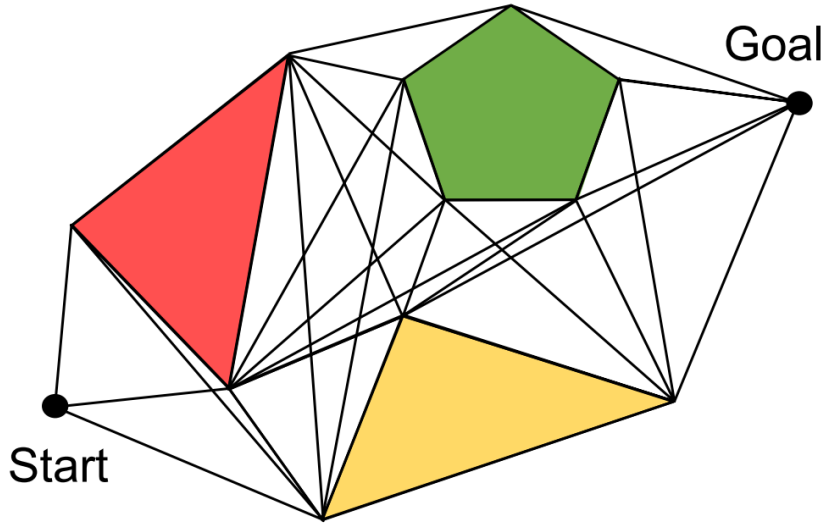
3. Path Planning for $Q = R^2$

The Visibility Graph

- A visibility graph is a graph whose vertices can “see” each other, i.e., edges in the graph correspond to paths that do not intersect the interior of any obstacle; the ‘line of sight” between adjacent vertices is visible.

- Assumption: the robot is a point in 2D planar space and obstacles are 2D polygons.

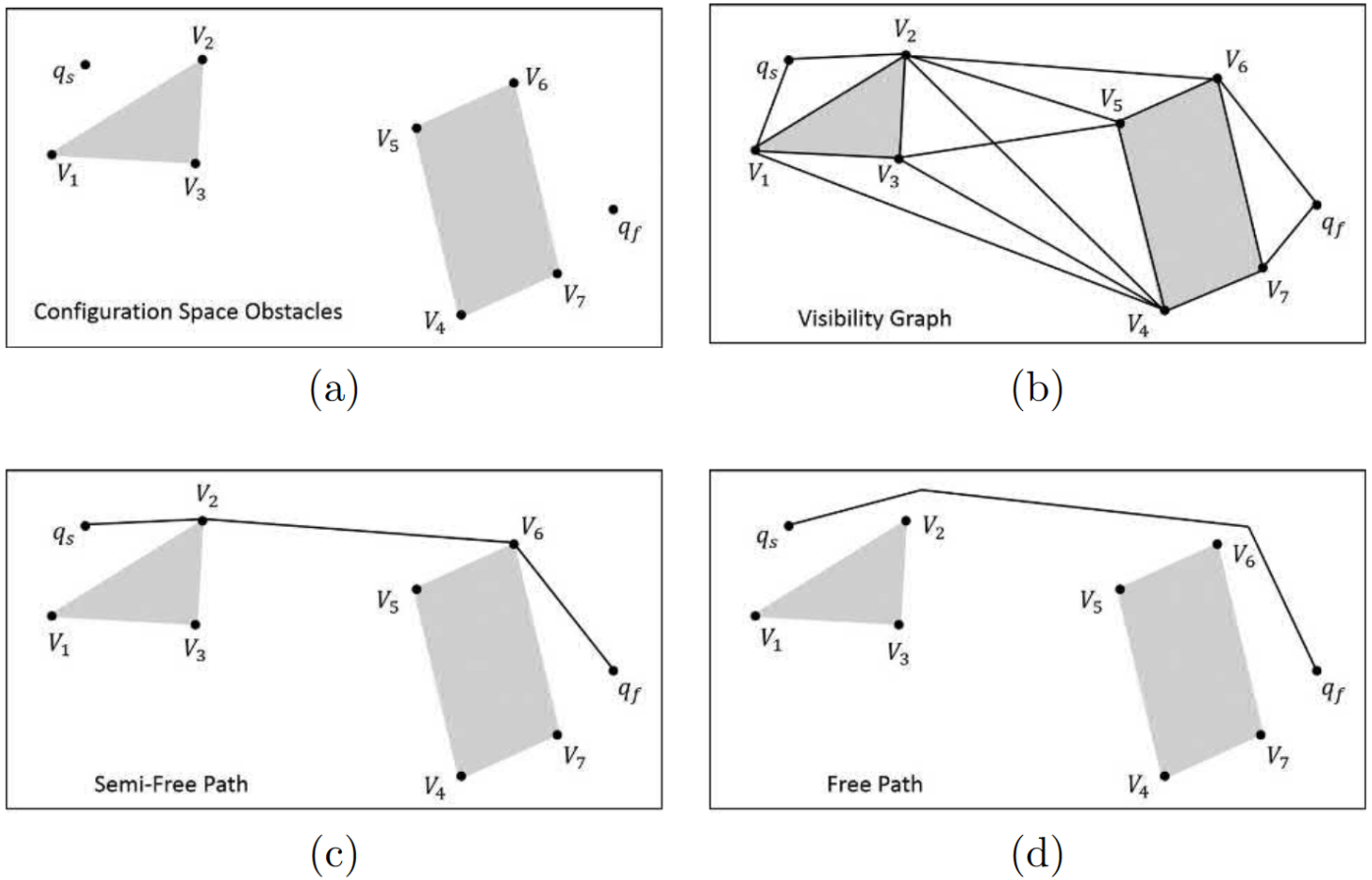
- If the line segment connecting two locations does not pass through any obstacle, an edge is drawn between them in the graph.



Visibility graph

3. Path Planning for $Q = R^2$

☐ The Visibility Graph



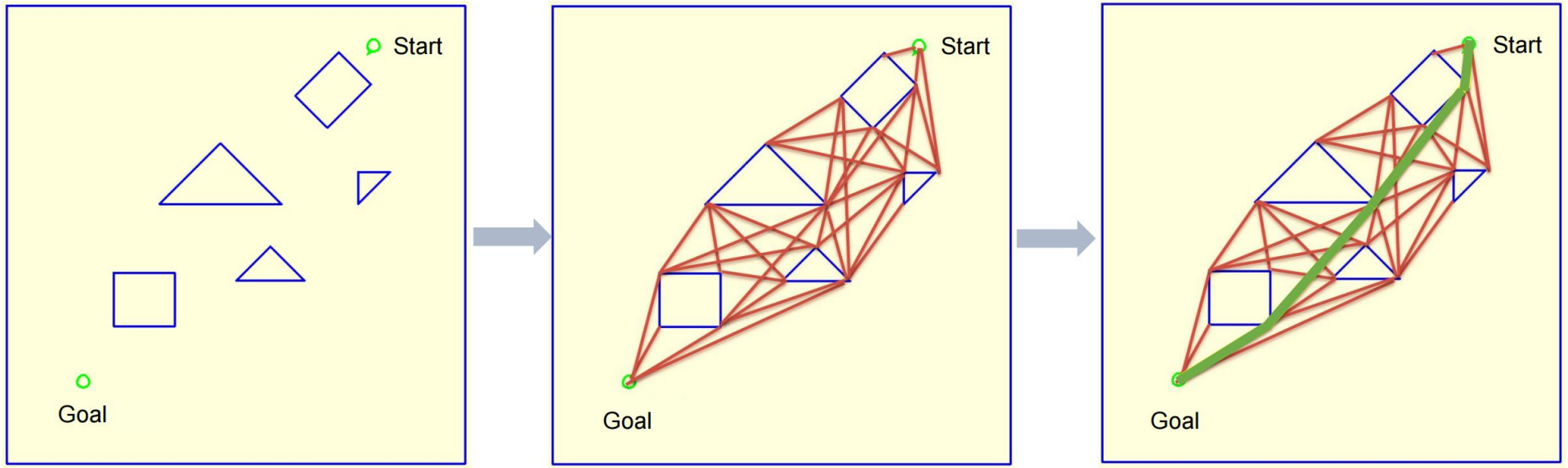
(a) An environment that contains polygonal obstacles, with start and goal configurations q_s and q_f. (b) The visibility graph. (c) A semi-free path from q_s to q_f. (d) A free path from q_s to q_f, obtained by a small perturbation of the path vertices that contact obstacle vertices for the semi-free path.

- **Semi-free path** means that the path lies in the envelope of the free configuration space, i.e., either in the collision-free configuration space, or in the boundary of the configuration space obstacle region.
- Deform a semi-free path into a free path by a **small perturbation** of the semi-free path, generate the free path.

Shortest path problem of Graph
https://en.wikipedia.org/wiki/Shortest_path_problem

3. Path Planning for $Q = R^2$

☐ Example



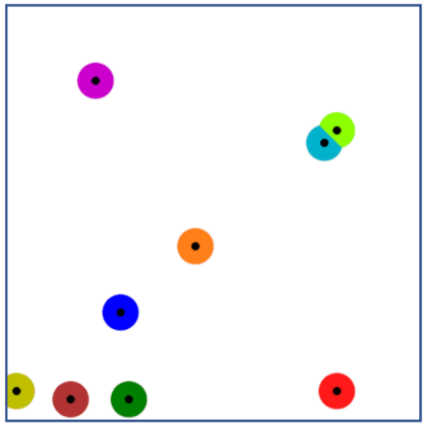
Problem | Construction of visibility graph | Shortest path between edges

Such shortest paths may **not be desirable in practice**, since they bring the robot **near to obstacles** in the workspace, which could easily lead to collision if there is any uncertainty in the robot's location or in the map of the environment.

Maximize the clearance between the robot and any obstacles ➡ Generalized Voronoi diagram

3. Path Planning for $Q = R^2$

(2) The Generalized Voronoi Diagram



Dot with color

Voronoi diagram

- In the whole of 2D space, follow the rule that the color of any given point equals the color of the nearest dot
- A Voronoi diagram is a partition of a plane into regions close to each of a given set of objects

3. Path Planning for $Q = R^2$

The Generalized Voronoi Diagram

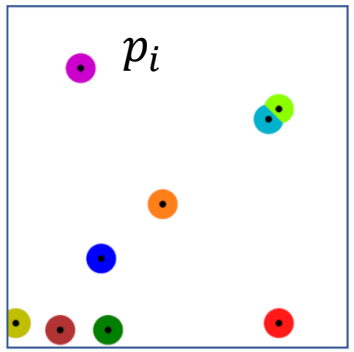
- Consider a set of discrete points in the plane, $P = \{p_1, \ldots, p_n\}$. For each point p_i , define its **Voronoi cell** to be the set of points that are closer to p_i than to any other $p_i \in$ P ,

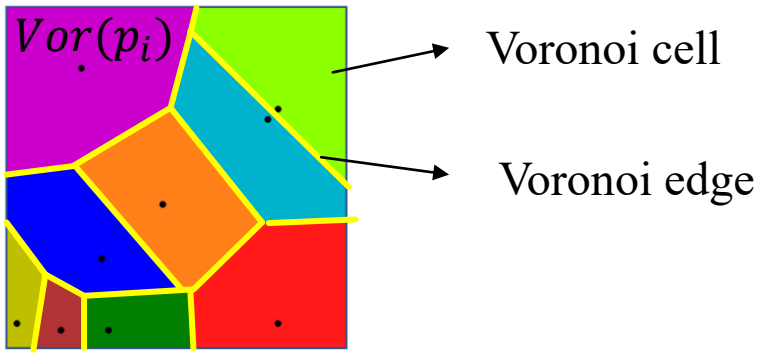
$$\mathrm{Vor}(p_i) = \{x \in \mathbb{R}^2 \mid \|x - p_i\| \leq \|x - p_j\|, \text{ for all } j \neq i\}$$

- Two Voronoi regions are adjacent if $\mathrm{Vor}(p_i) \cap \mathrm{Vor}(p_j) \neq \emptyset$, in which case the intersection is a straight-line segment, referred to as a **Voronoi edge**, defined as

$$E_{ij} = \{x \in \mathbb{R}^2 \mid \|x - p_i\| = \|x - p_j\| \leq \|x - p_k\|, \text{for all } k \neq i, j\}$$

(Minimally equidistant to the points p_i and p_j)

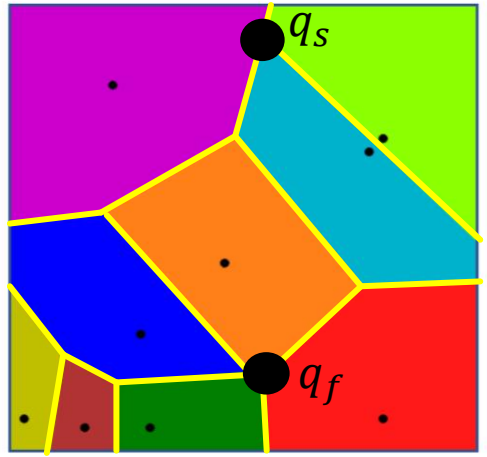


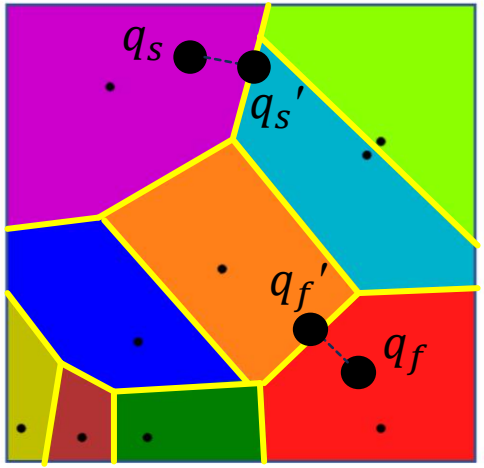


3. Path Planning for $Q = R^2$

The Generalized Voronoi Diagram

- If q_s and q_f lie in Voronoi edges, then the **maximum clearance path** connecting q_s and q_f is contained completely in the set of Voronoi edges.
  - Define the generalized Voronoi diagram as the graph $G = (V, E)$: edges are these Voronoi edges; vertices are points at which multiple Voronoi edges intersect.
  - As vertexes, add q_s and q_f to the Graph G, update the edge, find a path in the graph G from q_s and q_f.





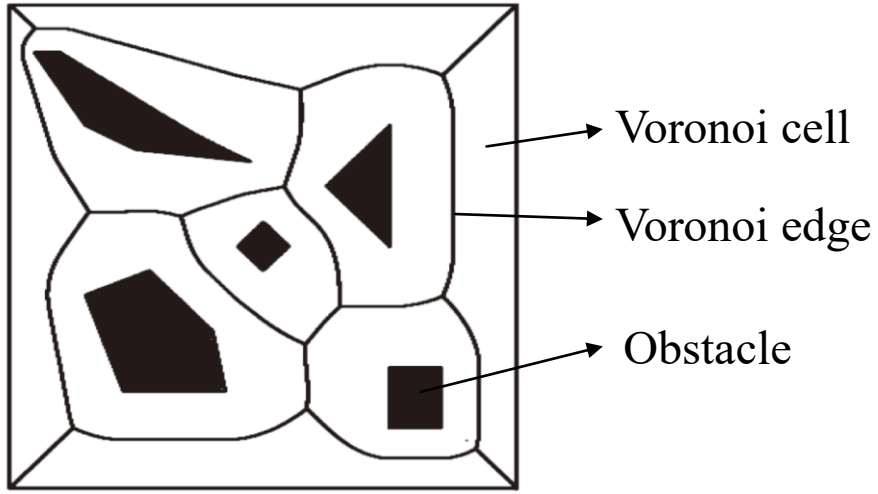
- If q_s and q_f do not lie in Voronoi edges
  - Find the straight-line path from q_s to the nearest Voronoi edge. Let q_s' denote the point at which this path intersects the Voronoi edge.
  - Find the straight-line path from q_f to the nearest Voronoi edge, get q_f'.
  - As vertexes, add q_s' to q_f' to the Graph G, update the edge, find a path in from q_s' to q_f'.

3. Path Planning for $Q = R^2$

☐ The Generalized Voronoi Diagram

- Generalize from the case of discrete points (obstacle) to the case of polygonal obstacles in the path planning process

Discretized points
as obstacle



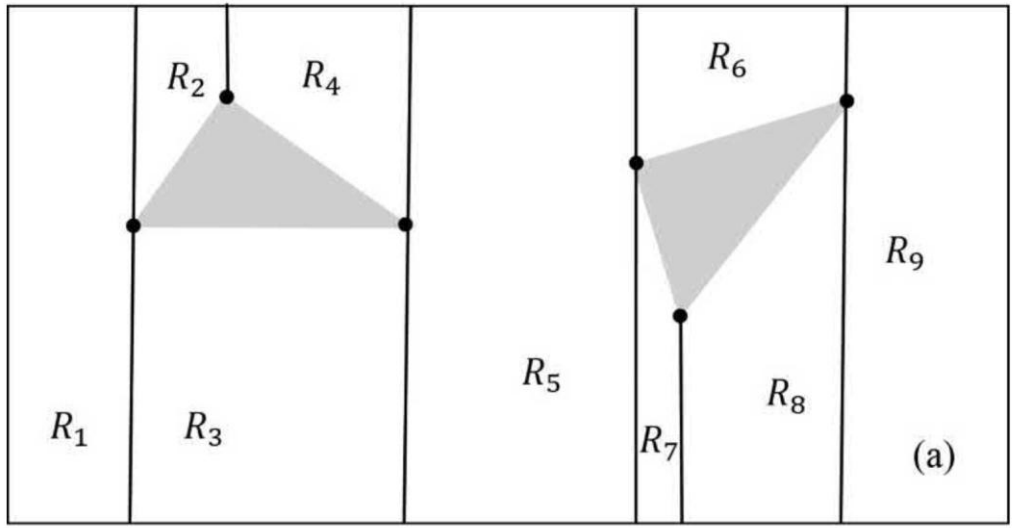
A polygonal configuration space containing five obstacles,
and its generalized Voronoi diagram

3. Path Planning for $Q = R^2$

(3) Trapezoidal Decompositions

- For the **visibility graph**, edges correspond to portions of specific semi-free paths; for the **Voronoi diagram**, each Voronoi edge corresponds to a portion of a specific free path. Neither of these representations **explicitly captures any information about the geometry of the free configuration space**.

- A spatial decomposition of the **free configuration space** is a collection of regions $\{R_1, \ldots, R_m\}$, and $\cup R_i = Q_{free}$, and there is no overlap for these regions.
- A trapezoidal decomposition is a special case from the class convex spatial decomposition that applies to the case of polygons in the plane.
- Trapezoidal decomposition is realized by sweeping a vertical line across the configuration space.



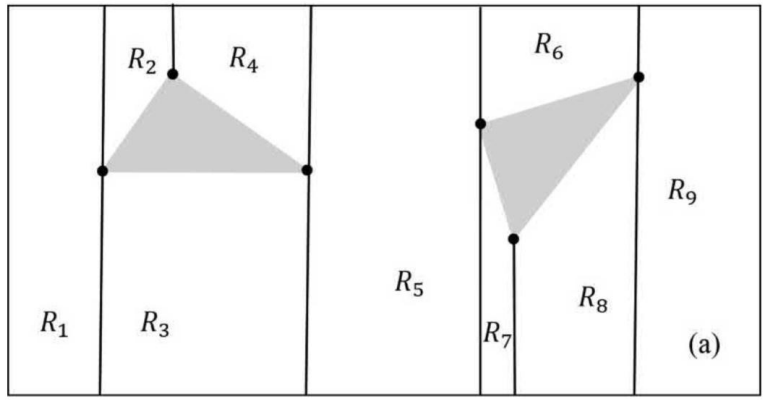
Each region in the decomposition is a trapezoid consisting of polygon edges and vertical line segments incident upon vertices of the polygons

3. Path Planning for $Q = R^2$

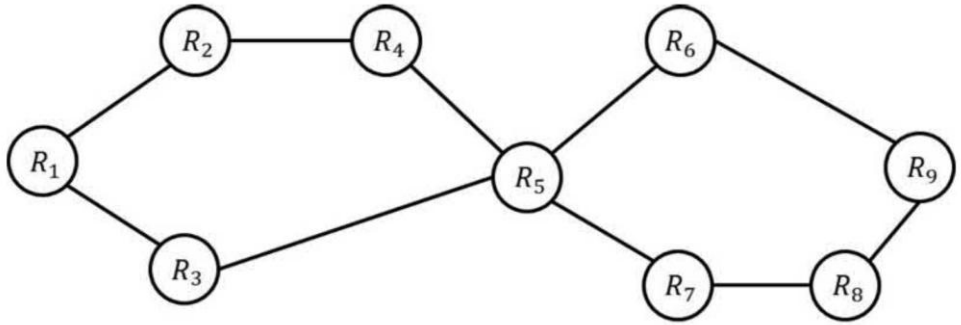
☐ Trapezoidal Decompositions

- **Connectivity graph** for the trapezoidal decomposition.

Trapezoidal decompositions



Connectivity graph

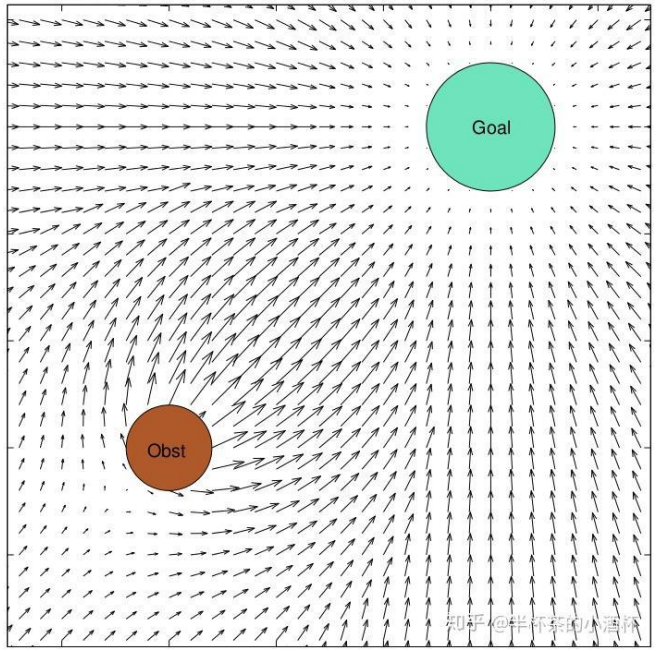


- The path planning problem can be solved as follows.
  - Determine the region R_i that contains the initial configuration q_s .
  - Determine the region R_j that contains the final configuration q_f .
  - Find a path in the connectivity graph from R_i to R_j .
  - Construct a piecewise linear path from q_s and q_f that passes successively through the cells found in Step 3, crossing from one region to the next at **the midpoint of the boundary of the two regions.**

4. Artificial Potential Fields

Introduction

- The path planning for $\boldsymbol{Q}=\boldsymbol{R^2}$ are applicable only for simple configuration spaces.
- For more complex configuration spaces (e.g., $SE(3)$ for a drone, or the n-torus for an n-link arm) is typically not feasible to build an explicit representation of $\mathcal{QO}$ or of $\mathcal{Q}_{free}$.
- The **artificial potential field** U is constructed so that the robot is **attracted** to the final configuration q_f while being **repelled** from the boundaries of $\mathcal{QO}$.



Additive field consisting of one attracts the robot to q_f and a second repels the robot from the boundary of $\mathcal{QO}$

$$U(q) = U_{att}(q) + U_{rep}(q)$$

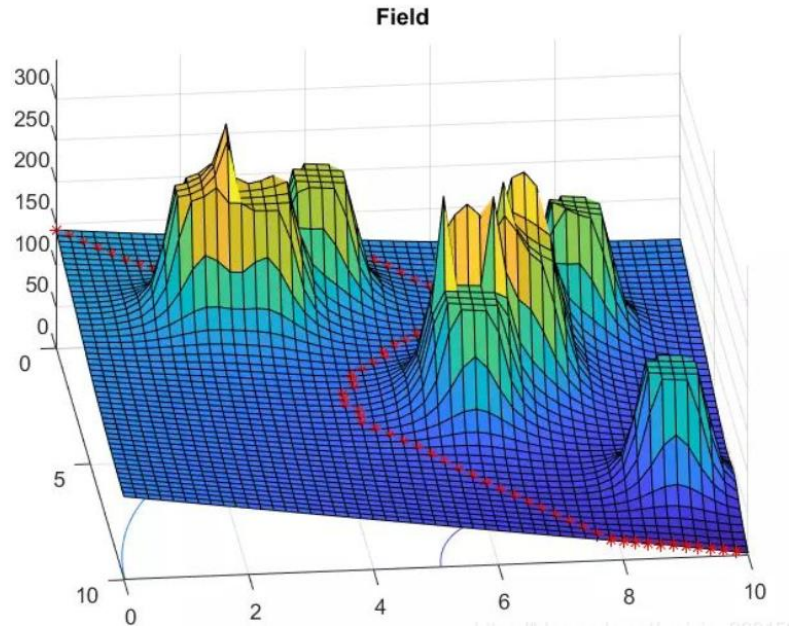
Attract the robot to q_f Repel the robot from the boundary of $\mathcal{QO}$

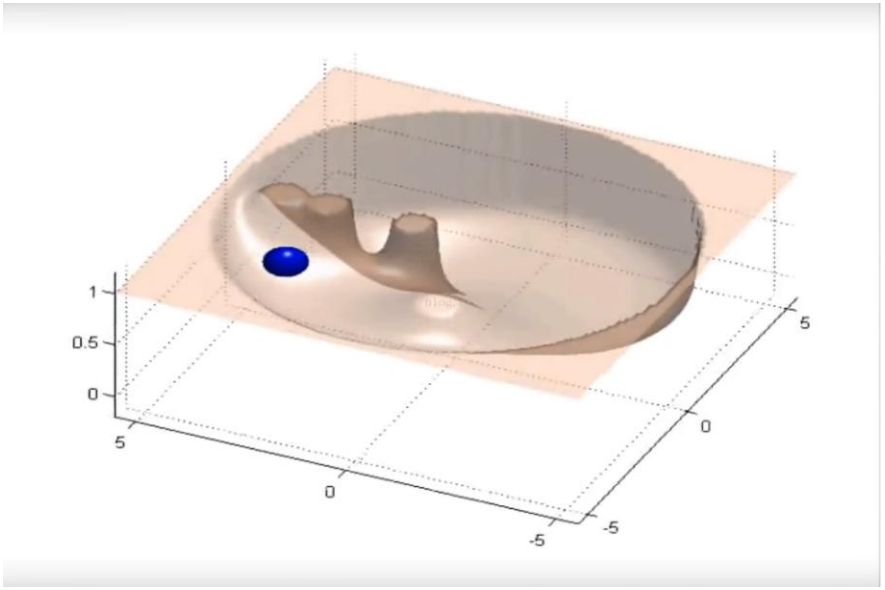
If possible, U is constructed so that there is a **single global minimum** of U at q_f and there are **no local minima**. Difficult or even impossible to construct such a field.

4. Artificial Potential Fields

Introduction

- **Formulation path planning as an optimization problem**: finding the global minimum of U starting from initial configuration q_s
  - **Gradient descent methods** are often used to find a solution: analogous to a particle moving under the influence of the force $F = -\nabla U$; attractive and repulsive forces when defining potential fields.





4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q = R^n$ - Attractive Field

- U_{att} should be monotonically increasing with the distance to the goal configuration.
- The simplest choice for such a field grows linearly with this distance, a so-called **conic well potential**:

$$U_{att}(q) = \zeta \|q - q_f\|$$

ζ is a parameter used to scale the effects of the attractive potential.

The **gradient of such a field has unit magnitude** everywhere except the goal configuration, where it is zero - lead to stability problems **since there is a discontinuity in the attractive force at the goal position**.

- The **parabolic well potential**: $U_{att}(q) = \frac{1}{2}\zeta \|q - q_f\|^2$

The parabolic well the attractive force is a vector directed toward q_f with magnitude linearly related to the distance to q from q_f.

$$F_{att}(q) = -\nabla U_{att}(q) = -\zeta(q - q_f)$$

But if q_s is very far from q_f , this may produce an **initial attractive force that is very large**.

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q = R^n$ - Attractive Field

- Combine the quadratic and conic potentials:

$$U_{att}(q) = \begin{cases} \frac{1}{2}\zeta \|q - q_f\|^2 & : \|q - q_f\| \le d \quad \text{Quadratic potential} \\ d\zeta \|q - q_f\| - \frac{1}{2}\zeta d^2 & : \|q - q_f\| > d \quad \text{Conic potential} \end{cases}$$

d is the distance that defines the transition from conic to parabolic well.

$$F_{att}(q) = \begin{cases} -\zeta(q - q_f) & : \|q - q_f\| \le d \\ -d\zeta \frac{(q - q_f)}{\|q - q_f\|} & : \|q - q_f\| > d \end{cases}$$

At the boundary $d = \|q - q_f\|$, the value and gradient of the quadratic potential is equal to the gradient of the conic potential $F_{att}(q) = -\zeta(q - q_f)$.

4. Artificial Potential Fields

Artificial Potential Fields for $Q = R^n$ - Repulsive Field

- Repel the robot from obstacles, never allowing the robot to collide with an obstacle.
- When the robot is far away from an obstacle, that obstacle should exert little or no influence on the motion of the robot.

The distance from configuration q to the boundary of $\mathcal{QO}$: $\rho = (q) \min_{q' \in \partial \mathcal{QO}} \|q - q'\|$

$$U_{rep}(q) = \begin{cases} \frac{1}{2}\eta\left(\frac{1}{\rho(q)} - \frac{1}{\rho_0}\right)^2 & : \rho(q) \le \rho_0 \\ 0 & : \rho(q) > \rho_0 \end{cases}$$

ρ_0 is the **distance of influence of an obstacle**, η is a parameter used to scale the effects of the repulsive potential.

$$F_{rep}(q) = \eta\left(\frac{1}{\rho(q)} - \frac{1}{\rho_0}\right)\frac{1}{\rho^2(q)}\nabla\rho(q): \quad \rho(q) \le \rho_0$$

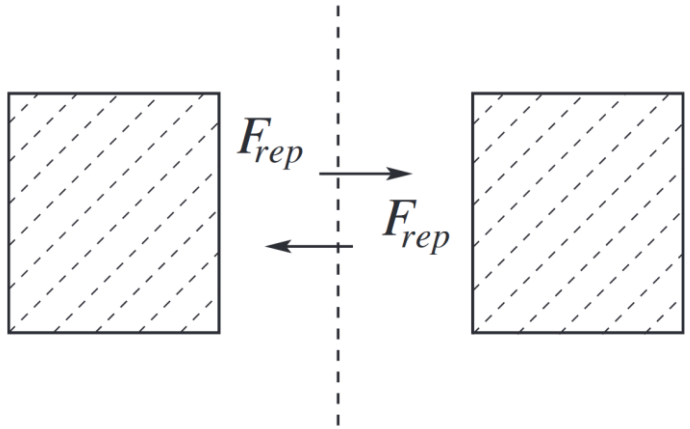
If $\mathcal{QO}$ is convex and b is the point on the boundary of $\mathcal{QO}$ that is closest to q, then $\rho(q) = \|q - b\|$, and its gradient is

$$\nabla\rho(q) = \frac{q - b}{\|q - b\|}$$

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q = R^n$ - Repulsive Field

If $\mathcal{QO}$ is **non-convex** or **two obstacles are close**, then the distance function ρ will not necessarily be differentiable everywhere, which implies discontinuity in the force vector.



The gradient changes discontinuously when q crosses the midline (gradient vanishing on the midline) between the two obstacles.

Deal with this problem: the simplest way is to ensure that the regions of influence of distinct obstacles **DOES NOT** overlap.

4. Artificial Potential Fields

Artificial Potential Fields for $Q = R^n$ - Gradient Descent Planning

- Starting at the **initial configuration**, take a small step in the direction of the **negative gradient** (which is the direction that decreases the potential as quickly as possible).
- This gives a new configuration q^{i+1}, and the process is repeated until the final configuration is reached.

$$q^{i+1} = q^i - \alpha^i \nabla U(q^i)$$

The scalar parameter α^i scales the **step size** at the i^{th}iteration. The choice for α^i is often made empirical basis, for example, based on the distance to the nearest obstacle or to the goal.

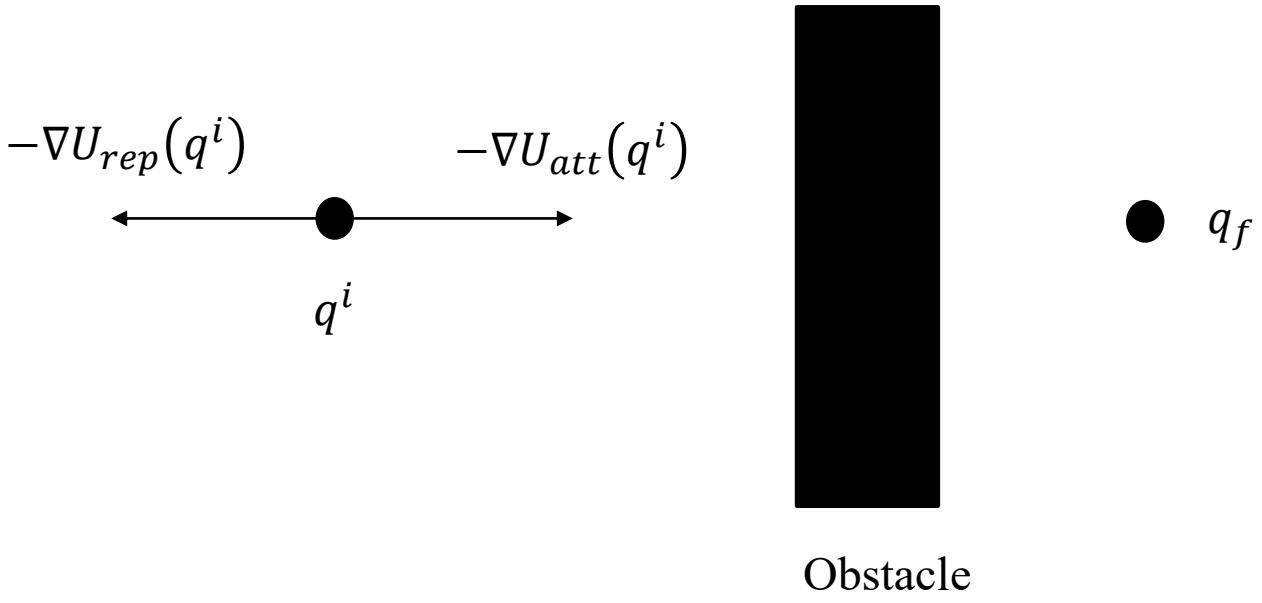
- Difficult to exactly satisfy the condition $q^i = q_f$, termination of the iteration:

$$\|q^i - q_f\| < \epsilon$$

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q = R^n$ - Escaping Local Minima

- The gradient descent algorithm is guaranteed to converge to a minimum in the field, but there is no guarantee that this minimum will be the global minimum: no guarantee that this method will find a path to q_f



In case of total gradient vanishing, the attractive force exactly cancels the repulsive force and the planner fails to make further progress.

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q = R^n$ - Escaping Local Minima

- **Randomized methods** have been developed to deal with this and other problems in robot motion planning.
  - **Determining when the planner is stuck in a local minimum**: if s**everal successive q^i lie within a small region** of the configuration space, it is likely that there is a nearby local minimum
  - **Defining the random walk to escape the local minimum**, e.g., simulate Brownian motion

$$q_{random-step} = (q_1 \pm v_1, \dots, q_n \pm v_n)$$

with v_i a fixed small constant and the probability of adding $+v_i$ or $-v_i$ equal to 1/2 (that is, a **uniform distribution**).

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q = R^n$ - Escaping Local Minima

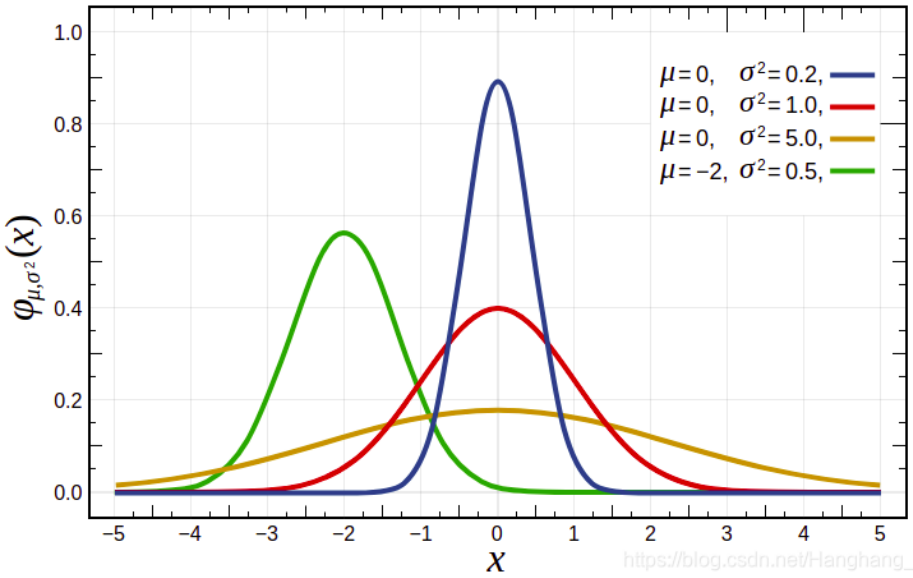
More general case for random walk: use probability theory to characterize the behavior of the random walk consisting of t random steps. T**he probability density function** for $q^t = (q_1, \dots, q_n)$:

$$p_i(q_i, t) \approx \frac{1}{v_i\sqrt{2\pi t}} \exp\left(-\frac{q_i^2}{2v_i^2 t}\right)$$

Gaussian density function

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- The variance $v_i^2\ t$ essentially determines the range of the random walk.
- If certain characteristics of local minima (for example, the size of the basin of attraction) are known in advance, these can be used to select the parameters v_i and t.

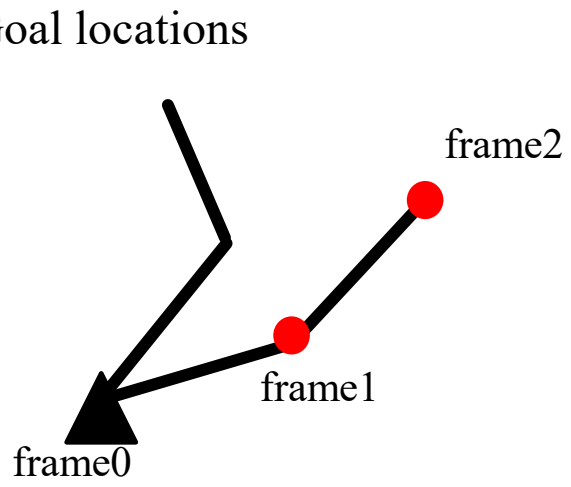


4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$

- The case of $Q \neq R^n$, difficult to construct a potential field directly on the configuration space, and difficult to compute the gradient of such a field, because of:
  - difficulty of computing shortest distances to configuration space obstacles.
  - complex geometry of the configuration space itself.
  - computational cost of computing an explicit representation of the boundary of the configuration space obstacle region.

- **For $Q \neq R^n$: define workspace potential fields** directly on the workspace of the robot, and then **map the gradients of these fields** to a corresponding **gradient for a configuration space potential function**.



For an *n*-link arm, define a **potential field for each of the origins** of the *n* DH frames (excluding the fixed - frame 0), these workspace potential fields will **attract the origins of the DH frames to their goal locations while repelling them from obstacles.**

Define **control points** on the robot that are sufficient to fully constrain its position

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$ - Attractive Field

- Define an attractive potential field $U_{att,i}(q)$ for o_i, the origin of the i-th DH frame: when all n origins reach their goal positions, the arm will have reached its goal configuration.
- Similarly, combine the conic and parabolic well potentials:

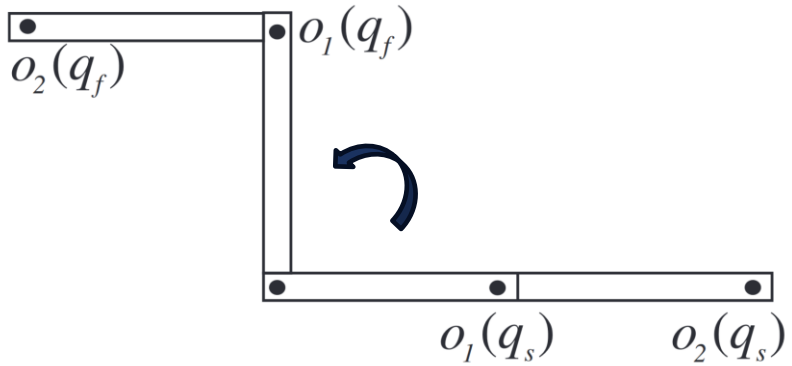
$$U_{att,i}(q) = \begin{cases} \frac{1}{2}\zeta_i \left\| o_i(q) - o_i(q_f) \right\|^2 & : \left\| o_i(q) - o_i(q_f) \right\| \leq d \\ d\zeta_i \left\| o_i(q) - o_i(q_f) \right\| - \frac{1}{2}\zeta_i d^2 & : \left\| o_i(q) - o_i(q_f) \right\| > d \end{cases}$$

d is the distance that defines the transition from conic to parabolic

- Workspace force for o_i is given by

$$F_{\text{att},i}(q) = \begin{cases} -\zeta_i (o_i(q) - o_i(q_\text{f})) & : \ ||o_i(q) - o_i(q_\text{f})|| \leq d \\ -d\zeta_i \dfrac{(o_i(q) - o_i(q_\text{f}))}{||o_i(q) - o_i(q_\text{f})||} & : \ ||o_i(q) - o_i(q_\text{f})|| > d \end{cases}$$

The gradient is well-defined at the boundary of the two fields



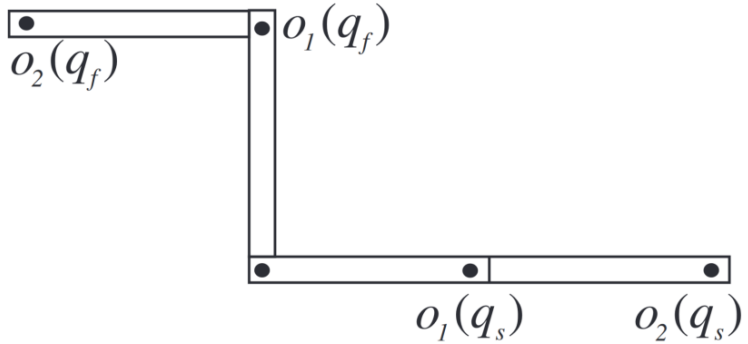
The initial configuration for the two-link arm is given by $\theta_1 = \theta_2 = 0$ and the final configuration is given by $\theta_1 = \theta_2 = \pi/2$. The origins for DH frames 1 and 2 are shown at both q_s and q_f

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$ - Attractive Field

- **Ex**: (Two-Link Planar Arm) Consider the two-link planar arm with $a_1 = a_2 = 1$ and with initial and final configurations given by

$$q_s = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \qquad q_f = \begin{bmatrix} \frac{\pi}{2} \\ \frac{\pi}{2} \end{bmatrix}$$



Using the forward kinematic equations for this arm we obtain:

$$o_1(q_s) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad o_1(q_f) = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad o_2(q_s) = \begin{bmatrix} 2 \\ 0 \end{bmatrix} \quad o_2(q_f) = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

Using these coordinates for the origins of the two DH frames at their initial and goal configurations, assuming that d is sufficiently large, we obtain the attractive forces.

$$F_{att,1}(q_s) = -\zeta_1(o_1(q_s) - o_1(q_f)) = \zeta_1 \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

$$F_{att,2}(q_s) = -\zeta_2(o_2(q_s) - o_2(q_f)) = \zeta_2 \begin{bmatrix} -3 \\ 1 \end{bmatrix}$$

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$ - Repulsive Field

- To prevent collisions, define a workspace repulsive potential field for the origin of each DH frame (excluding frame 0)

The distance in the workspace from the origin of DH frame *i* to the nearest obstacle:

$$\rho(o_i(q)) = \min_{x \in \partial Q} \|o_i(q) - x\|$$

Workspace repulsive potential is:

$$U_{rep,i}(q) = \begin{cases} \frac{1}{2}\eta_i \left(\frac{1}{\rho(o_i(q))} - \frac{1}{\rho_0}\right)^2 & : \rho(o_i(q)) \leq \rho_0 \\ 0 & : \rho(o_i(q)) > \rho_0 \end{cases}$$

ρ_0: workspace distance of influence of an obstacle

Workspace repulsive force: $F_{rep,i} = (q)\eta_i(\frac{1}{\rho(o_i(q))} - \frac{1}{\rho_0})\frac{1}{\rho^2(o_i(q))}\nabla\rho(o_i(q))$

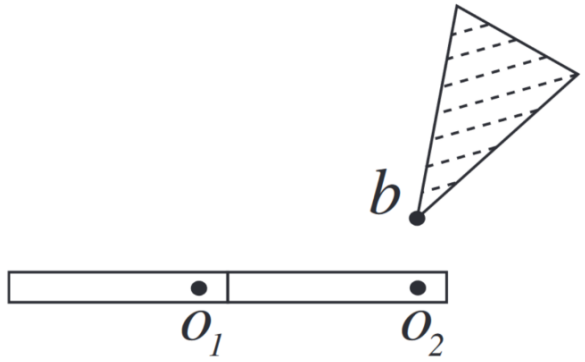
If the obstacle region is convex and b is the point on the obstacle boundary that is closest to o_i, then $\rho(o_i(q)) = \|o_i(q) - b\|$

$$\nabla\rho(x)|_{x=O_i(q)} = \frac{o_i(q) - b}{\|o_i(q) - b\|}$$

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$ - Repulsive Field

- **Ex**: (Two-Link Planar Arm) A single convex obstacle in the workspace as shown in below. Let $\rho_0 = 1$. This prevents the obstacle from repelling O_1, which is reasonable since link 1 can never contact the obstacle. The nearest obstacle point to O_2 is the vertex b of the polygonal obstacle. Suppose that b has the coordinates (2, 0.5). Then the distance from $O_2(q_s)$ to b is $\rho(O_2(q_s)) = 0.5$ and $\nabla\rho(O_2(q_s)) = [0, -1]^T$. The repulsive force at the initial configuration for O_2 is then given by



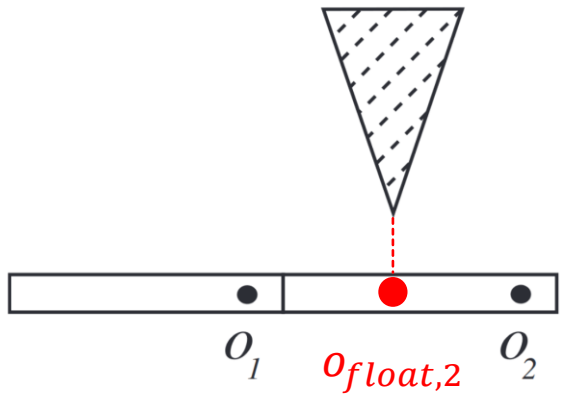
$$F_{\text{rep},2}(q_s) = \eta_2 \left(\frac{1}{0.5} - 1 \right) \frac{1}{0.25} \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \eta_2 \begin{bmatrix} 0 \\ -4 \end{bmatrix}$$

This force has no effect on joint 1, but causes joint 2 to rotate slightly in the clockwise direction, moving link 2 away from the obstacle.

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$ - Repulsive Field

- Defining repulsive fields **only for the origins of the DH frames** does not guarantee that the robot cannot collide with an obstacle.



The repulsive forces exerted on the origins of the DH frames o_1 and o_2 may not be sufficient to prevent a collision between link 2 and the obstacle

- The **floating control points** are defined as points on the boundary of a link that are closest to any workspace obstacle.

The choice of the $o_{float,2}$ depends on the configuration q

- The repulsive force acting on $o_{float,i}$ is defined in the same way as for the other control points.

$$F_{rep,i}(q) = \eta_i\left(\frac{1}{\rho\left(o_{float,i}(q)\right)} - \frac{1}{\rho_0}\right)\frac{1}{\rho^2\left(o_{float,i}(q)\right)}\nabla\rho\left(o_{float,i}(q)\right)$$

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$ - Mapping Workspace Forces to Joint Torques

- How these artificial forces can be mapped to **artificial joint torques**.
- According to the principle of **virtual work and static force of robotics**, if τ denotes the vector of joint torques induced by the workspace force F exerted at the end effector, then:

$$J_v^T F = \tau$$

J_v includes the top three rows of the manipulator Jacobian.

- we have considered only attractive and repulsive workspace forces, and not attractive and repulsive workspace torques.
- For each o_i, an appropriate Jacobian must be constructed, denoted as J_{O_i}.

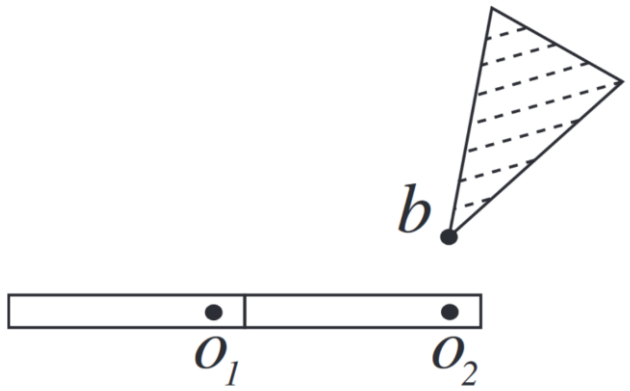
4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$ - Mapping Workspace Forces to Joint Torques

- **Ex**: (Two-Link Planar Arm) Consider again the two-link arm with workspace forces as given in the previous example (P32 and P34). The Jacobians that map joint velocities to linear velocities satisfy:

$$\dot{O}_i = J_{O_i}(q)\begin{bmatrix}\dot{q}_1 \\ \dot{q}_2\end{bmatrix}$$

For the two-link arm, the Jacobian matrix for O_2:



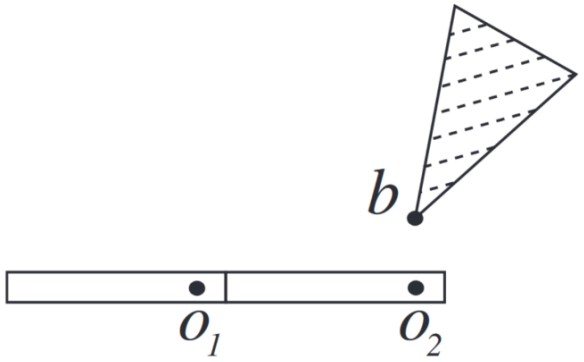
$$J_{o_2}(q_1, q_2) = \begin{bmatrix} -s_1 - s_{12} & -s_{12} \\ c_1 + c_{12} & c_{12} \end{bmatrix}$$

The Jacobian matrix for o_1 is similar, but takes into account that motion of joint 2 does not affect the velocity of o_1.

$$J_{o_1}(q_1, q_2) = \begin{bmatrix} -s_1 & 0 \\ c_1 & 0 \end{bmatrix}$$

4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$ - Mapping Workspace Forces to Joint Torques



At $q_s = (0, 0)$ we have $J_{o_1}^T(q_s) = \begin{bmatrix} -s_1 & c_1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$

and $J_{o_2}^T(q_s) = \begin{bmatrix} -s_1 - s_{12} & c_1 + c_{12} \\ -s_{12} & c_{12} \end{bmatrix} = \begin{bmatrix} 0 & 2 \\ 0 & 1 \end{bmatrix}$

Using these Jacobians, we can easily map the workspace attractive and repulsive forces to joint torques. If we let $\zeta_1 = \zeta_2 = \eta_2 = 1$ we obtain

$$\tau_{\text{att},1}(q_s) = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\tau_{\text{att},2}(q_s) = \begin{bmatrix} 0 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\tau_{\text{rep},2}(q_s) = \begin{bmatrix} 0 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -4 \end{bmatrix} = \begin{bmatrix} -8 \\ -4 \end{bmatrix}$$

4. Artificial Potential Fields

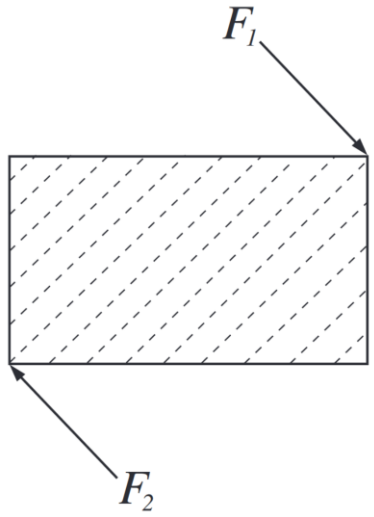
☐ Artificial Potential Fields for $Q \neq R^n$ - Mapping Workspace Forces to Joint Torques

- The total artificial joint torque acting on the arm is **the sum of the artificial joint torques** that result from all attractive and repulsive potentials.

$$\tau(q) = \sum_i J_{o_i}^T(q) F_{att,i}(q) + \sum_i J_{o_i}^T(q) F_{rep,i}(q)$$

- It is essential that we add the joint torques and not the workspace forces. In other words, we must u**se the Jacobians to transform forces to joint torques** before we combine the effects of the potential fields.

E.g., **vector addition** of these two forces produces zero net force, but there is a **net torque induced** by these forces.

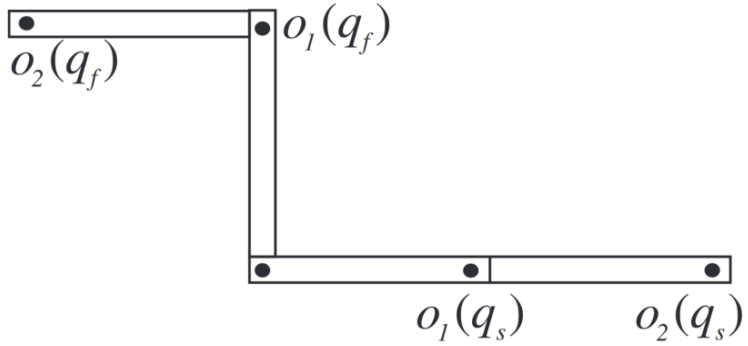


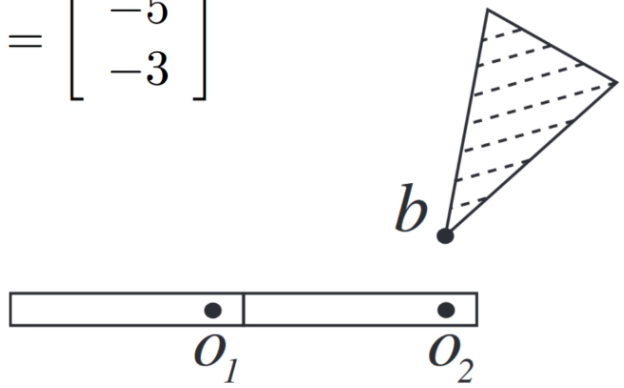
4. Artificial Potential Fields

☐ Artificial Potential Fields for $Q \neq R^n$ - Mapping Workspace Forces to Joint Torques

- **Ex**: (Two-Link Planar Arm) Consider again the two-link planar arm (P38), the total joint torque induced by the attractive and repulsive workspace potential fields is given by

$$\begin{aligned}\tau(q_s) &= \tau_{att,1}(q_s) + \tau_{att,2}(q_s) + \tau_{rep,2}(q_s) \\ &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} -8 \\ -4 \end{bmatrix} = \begin{bmatrix} -5 \\ -3 \end{bmatrix}\end{aligned}$$





These joint torques have the effect of **causing each joint to rotate in a clockwise direction**, away from the goal, due to the close proximity of o_2 to the obstacle. By choosing a smaller value for η_2, this effect can be overcome.

4. Artificial Potential Fields

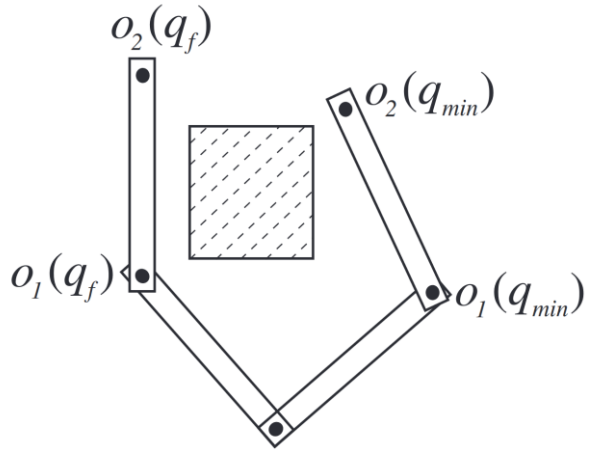
☐ Artificial Potential Fields for $Q \neq R^n$ - Gradient Descent Planning

- The gradient descent algorithm constructs a sequence of configurations, $q^0, q^1, \ldots, q^m$ such that $q^0 = q_s$ and $q^m = q_f$, but in this case the k^{th} iteration is given by:

$$q^{k+1} = q^k + \alpha^k \frac{\tau(q^k)}{\|\tau(q^k)\|} \quad \text{where, } \tau(q) = \sum_i J_{o_i}^T(q) F_{att,i}(q) + \sum_i J_{o_i}^T(q) F_{rep,i}(q)$$

The scalar α^k determines the step size at the k^{th} iteration, and the algorithm terminates when $\|q^k - q_f\| < \epsilon$.

- The problem of **local minima** in the potential field also exists for this approach to defining workspace potentials and mapping workspace gradients to the configuration space.

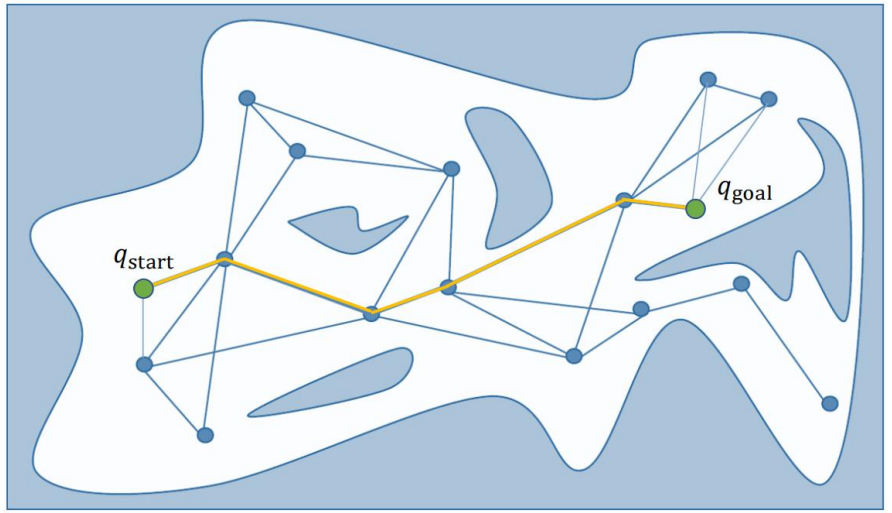


The configuration q_{min} is a local minimum in the potential field. At q_{min} the **attractive force exactly cancels the repulsive force** and the planner fails to make further progress

5. Sampling-Based Methods

☐ Introduction

- Potential field approaches is goal-driven (due to the attractive potential), susceptible to failure due to the presence of **local minima** in the potential field.
- Design a planner that completely **abandons goal-driven search**, instead relying completely on **randomized strategies - sampling-based planners**.
- The simplest such strategy is to generate **random samples** from a **uniform probability distribution** on the configuration space.
- Produces a graph $G = (V, E)$ in which the vertex set V includes the **randomly generated sample configurations**, edges correspond to local paths between sample configurations, graph G is referred to as a **configuration space roadmap.**

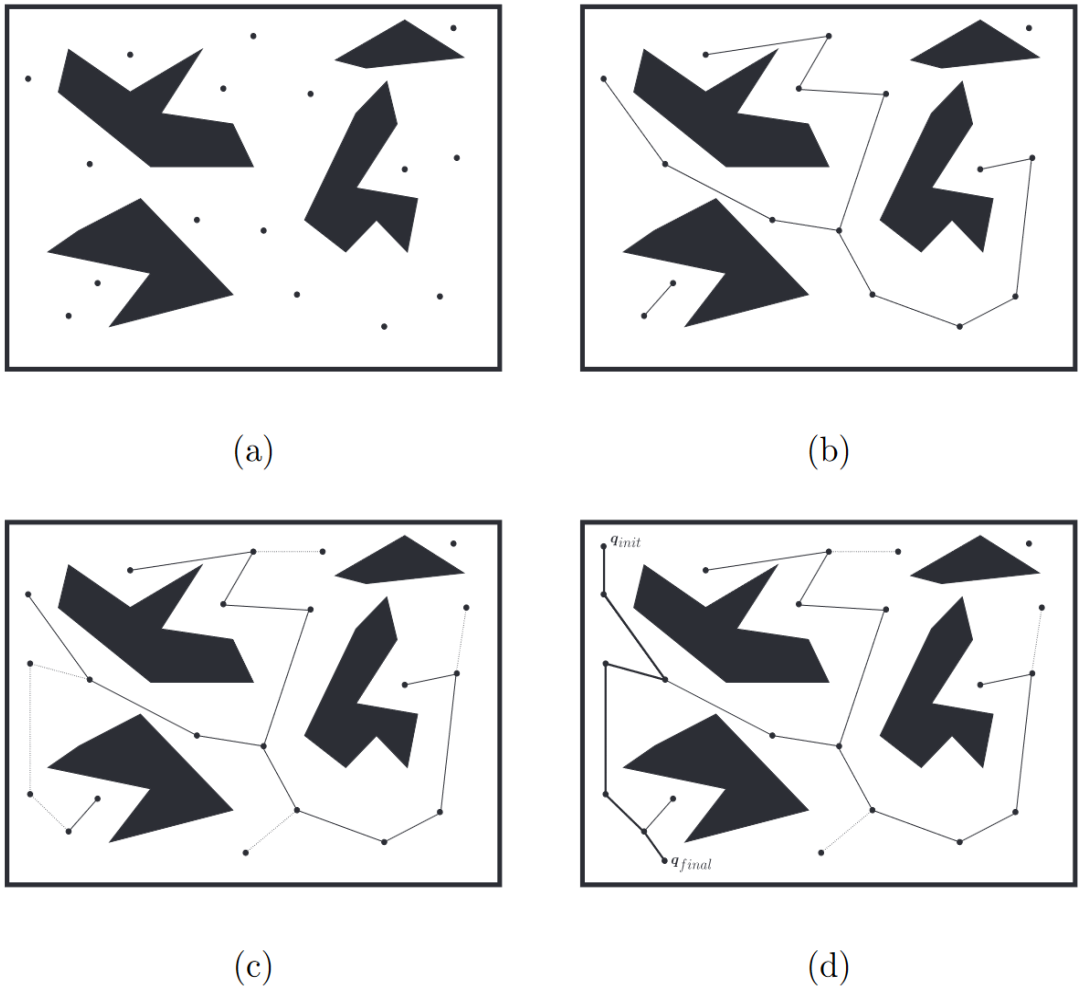


- **Two methods: Probabilistic roadmap (PRM), Rapidly-exploring Random Tree (RRT).**

5. Sampling-Based Methods

☐ Probabilistic Roadmaps (PRM) – General processes

- A set of **random samples is generated** in the configuration space, only collision-free samples are retained.
- **Each sample is connected to its nearest neighbors using a straight-line path**. If such a path causes a collision, the corresponding samples are not connected in the roadmap.
- **Additional samples are generated** and connected to the roadmap to address the **multiple connected components**.
- A path from q_s to q_f is found by **connecting q_s and q_f** to the roadmap and then **searching this augmented roadmap for a path from q_s to q_f** .

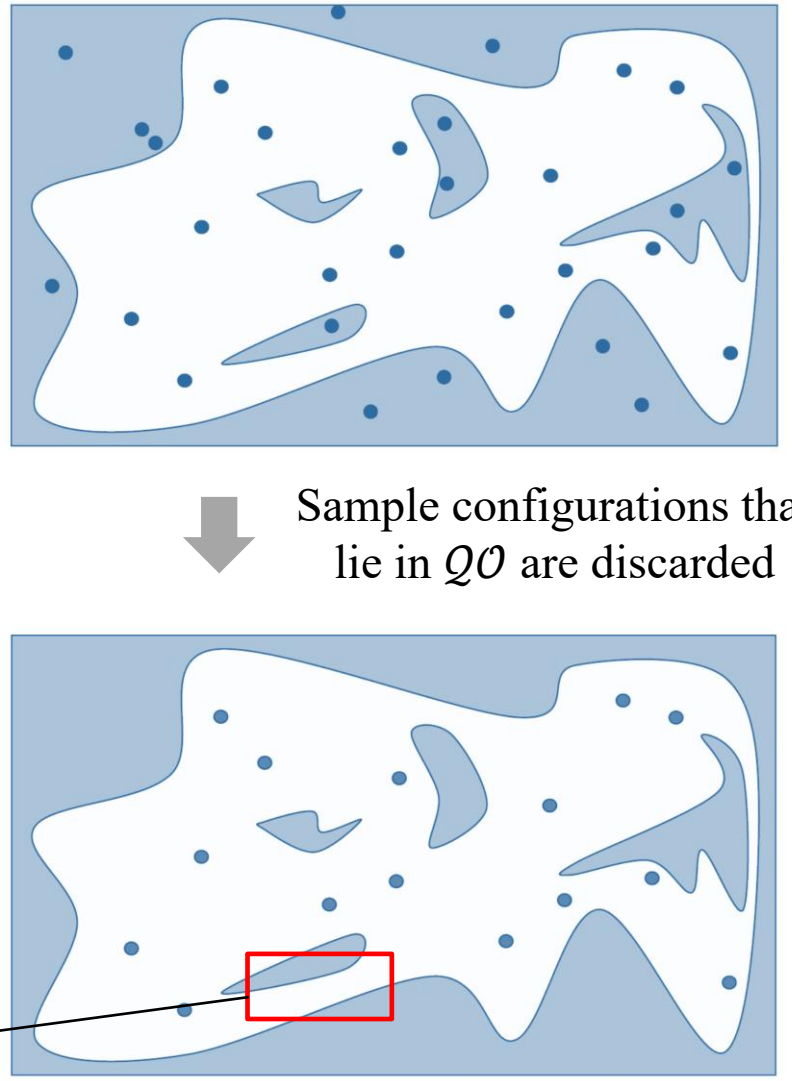


The steps in the construction of a probabilistic roadmap for a two-dimensional configuration space containing polygonal obstacles

5. Sampling-Based Methods

☐ Probabilistic Roadmaps (PRM) - Sampling the Configuration Space

- Generate sample configurations in the configuration space, e.g., uniform random sampling.
- Sample configurations that lie in $\mathcal{QO}$ are discarded.
- The number of samples in $\mathcal{Q}_{free}$ is proportional to the volume of the region.
- Uniform sampling is **unlikely to** place samples in narrow passage of $\mathcal{Q}_{free}$.



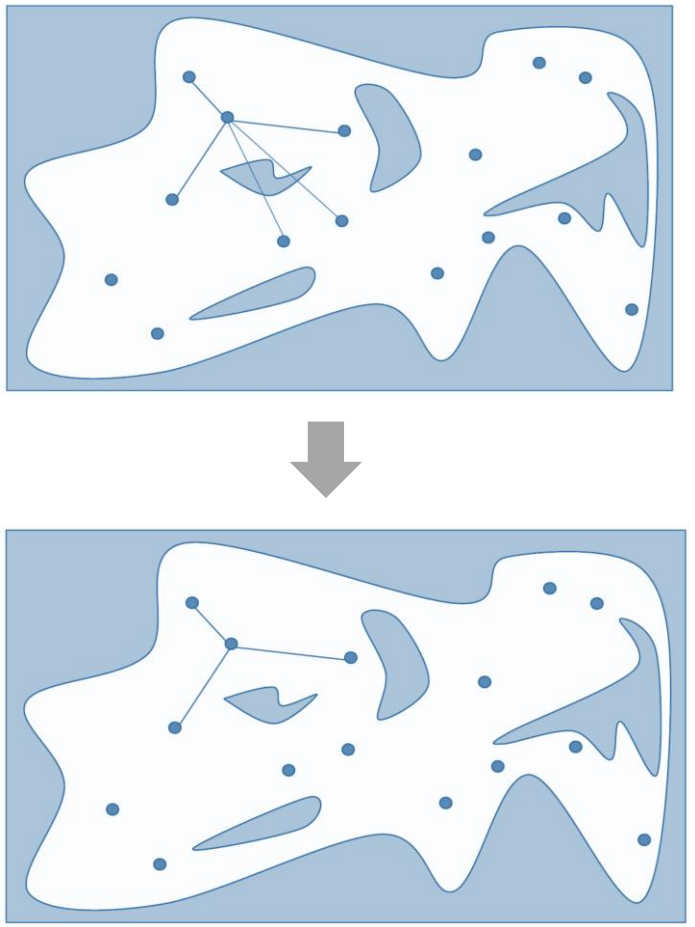
5. Sampling-Based Methods

☐ Probabilistic Roadmaps (PRM) - Connecting Pairs of Configurations

- Connect each vertex to its k nearest neighbors, e.g., $k = 5$, distance function is required, four commonly functions that have been popular.

2-norm in $\mathcal{Q}$:	$\|q' - q\| = \left[\sum_{i=1}^{n}(q'_i - q_i)^2\right]^{\frac{1}{2}}$
∞-norm in $\mathcal{Q}$:	$\max_n \|q'_i - q_i\|$
2-norm in workspace:	$\left[\sum_{\mathrm{p}\in\mathcal{A}}\|\mathrm{p}(q') - \mathrm{p}(q)\|^2\right]^{\frac{1}{2}}$
∞-norm in workspace:	$\max_{\mathrm{p}\in\mathcal{A}}\|\mathrm{p}(q') - \mathrm{p}(q)\|$

A is a set of reference points on the robot, and $p(q)$ refers to the workspace coordinates of reference point p at configuration q



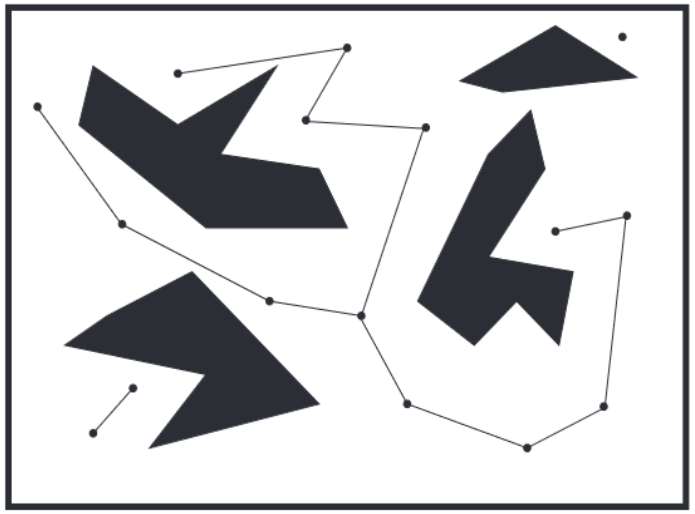
- Connect neighboring vertices, **keep the collision-free paths**. The simplest approach to collision detection is to sample the straight-line path at a sufficiently fine discretization, and check each sample for collision.

Use incremental collision detection approaches, collision detection software packages, e.g., OP-Code, or FCL

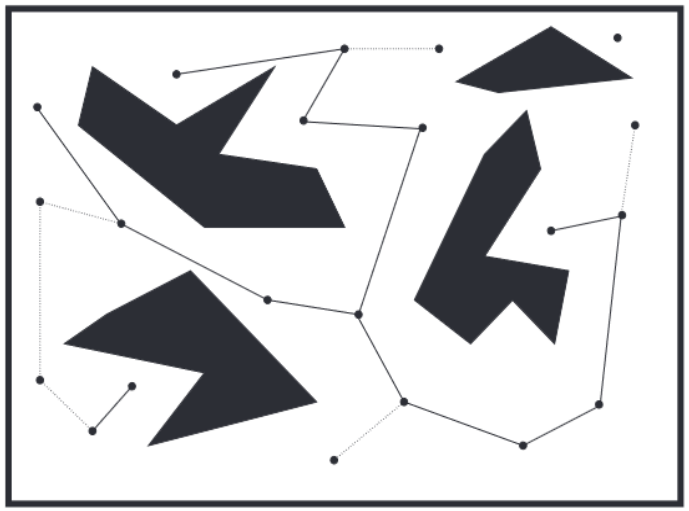
5. Sampling-Based Methods

☐ Probabilistic Roadmaps (PRM) - Enhancement

- In case of multiple connected components, **enhancement process** is to connect as many of these disjoint components as possible.
  - Attempt to connect pairs of vertices in two disjoint components.
    - (1) Identify the largest connected component, and connect the smaller components to it.
    - (2) Choose a vertex randomly as a candidate for connection, a vertex with fewer neighbors is a more likely to be candidate for connection.
  - Add more random vertices to the roadmap, in the hope of **finding vertices that lie in or near the narrow passages**: identify vertices that have few neighbors, generate sample configurations in regions around these vertices.



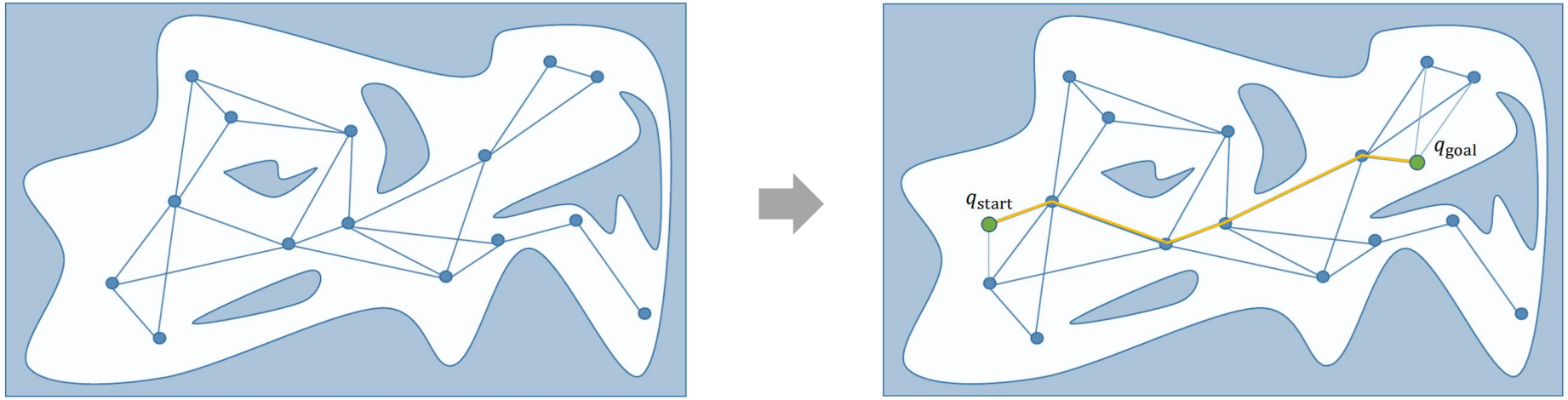
Add more random vertices



5. Sampling-Based Methods

☐ Probabilistic Roadmaps (PRM) – Form roadmap and plan a path

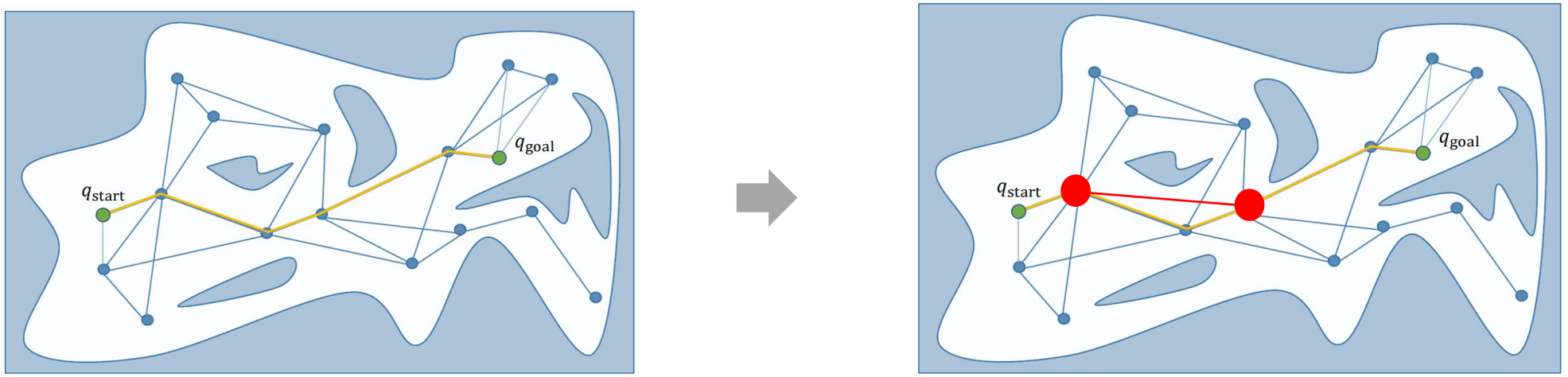
- The roadmap is an undirected graph.
- Include the start node q_{start} and goal node q_{goal} as the sampling points and plan a shortest path from the start node to the goal node. (Typically using A$*$ method)



5. Sampling-Based Methods

☐ Probabilistic Roadmaps (PRM) - Path Smoothing

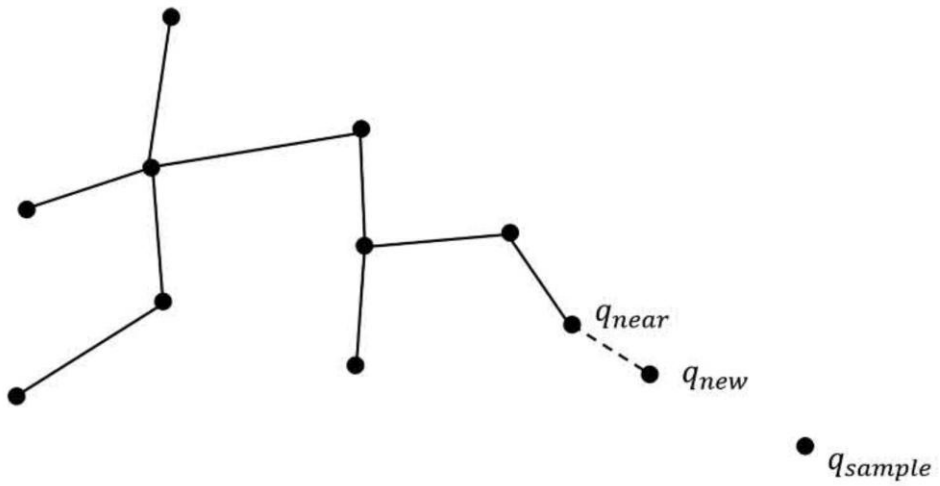
- The simplest path smoothing algorithm is to **select two random points on the path and try to connect them with line segment**, this process is repeated until no significant progress is made.



5. Sampling-Based Methods

☐ Rapidly-Exploring Random Trees (RRTs)

- In the PRM method, **new samples obtained uniformly from Q is independent of any previously generated samples**, could be far from existing vertices, likely causing connection failures and number of connected components.
- **Rapidly-Exploring Random Trees (RRTs)**: grow a single tree, starting from a vertex that corresponds to q_s , until some leaf vertex reaches the goal configuration.

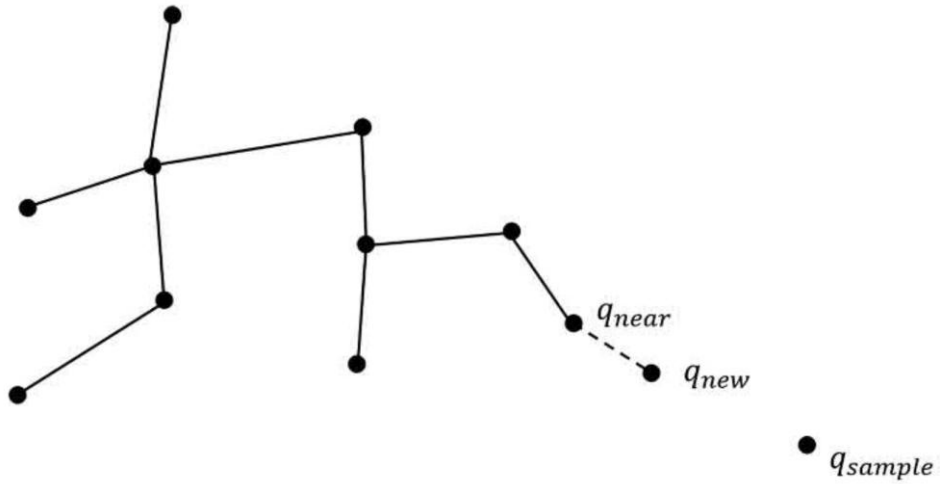


- For an existing tree, generating a sample configuration q_{sample} from a uniform probability distribution on Q.
- Identifying q_{near}, the vertex in the current tree that is nearest to q_{sample}.
- Taking a small step from q_{near} toward q_{sample} to generate q_{new}, add it to the tree; q_{new} is a new vertex in the tree.

5. Sampling-Based Methods

☐ Rapidly-Exploring Random Trees (RRTs)

- Initialization
- Random sampling
- Find the nearest point
- Generate new point
- Collision checking and tree expension
- Goal checking



Algorithm 1 RRT-Basic Algorithm

1: $T \leftarrow tree(x_{init})$;
2: For $k = 1$ *to* K do
3: $x_{rand} \leftarrow RANDOM\_CONFIG(C_{free})$;
4: $x_{near} \leftarrow NEAREST\_NEIBOR(T, x_{rand})$;
5: $x_{new} \leftarrow NEW\_CONFIG(x_{near}, \varepsilon, x_{rand})$;
6: If $COLLISION\_FREE(x_{near}, x_{new})$ then
7: $GROWN\_TREE(T, x_{new})$;
8: End If
9: If $\|x_{new} - x_{goal}\| < \rho_{min}$ then
10: return T;
11: End If
12: End For

5. Sampling-Based Methods

☐ Rapidly-Exploring Random Trees (RRTs)

- **Example on how to determine q_{new}**: a robot whose system dynamics $\dot{x} = f(x, u)$, and x denotes the state of the system and u denotes an input.

  Generate the vertex x_{new} by integrating the dynamics forward in time from the initial condition given by x_{near}:

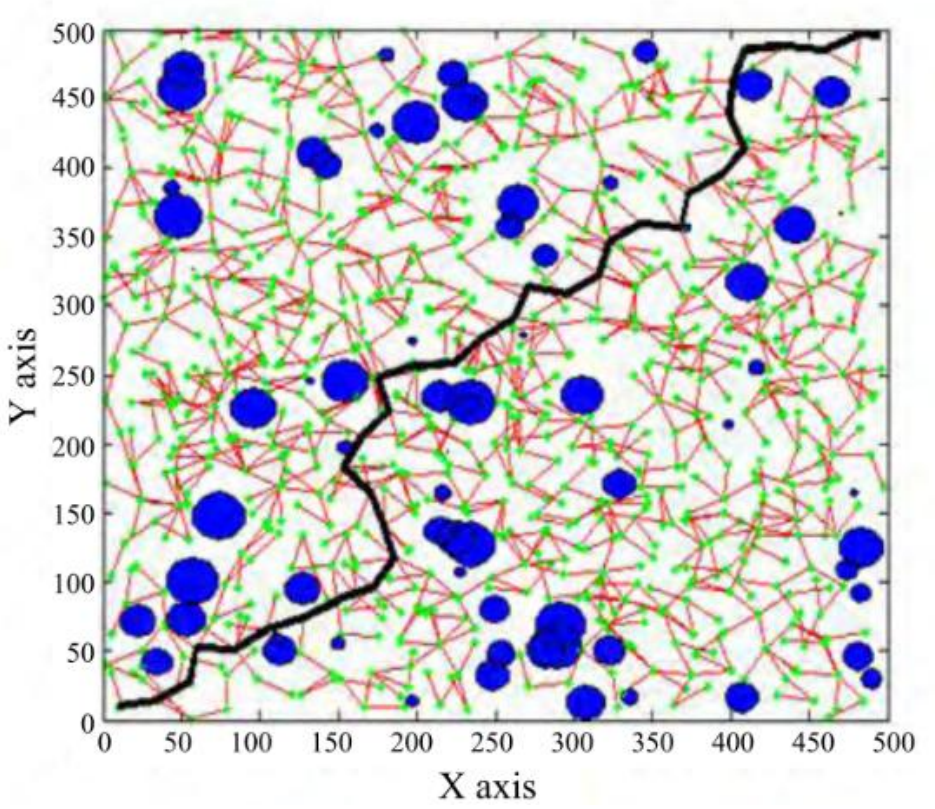
$$x_{new} = x_{near} + \int_0^{\Delta t} f(x, u)dt$$

  Choosing u to ensure that x_{new} lies along the line connecting $x_{near}$to x_{sample}: a popular way is to randomly select a set of candidate inputs, u^i, evaluate the above equation for each of these, and retain the result that makes the best progress toward x_{sample}.

5. Sampling-Based Methods

- **Rapidly-Exploring Random Trees (RRTs)**

An animation of an RRT starting from iteration 0 to 10000

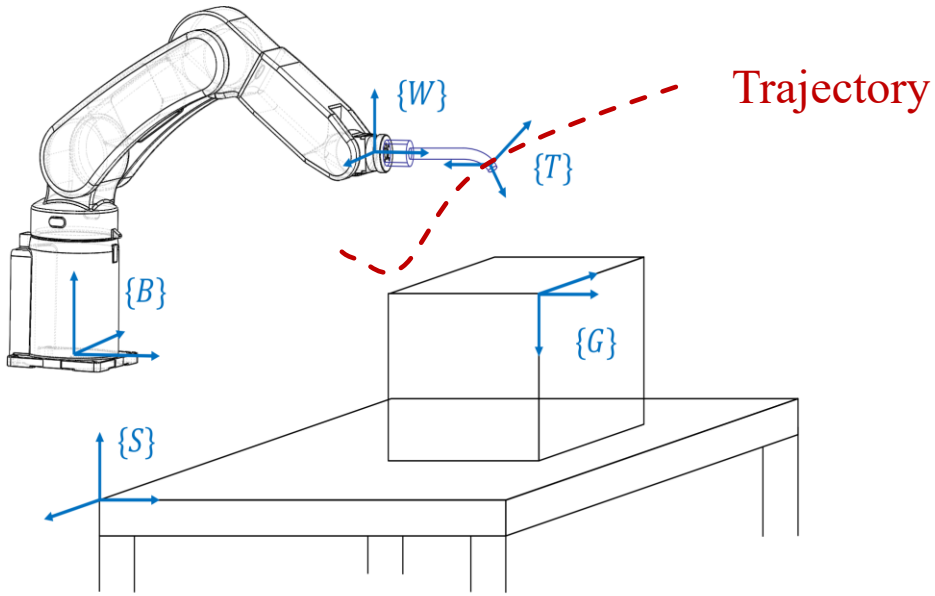


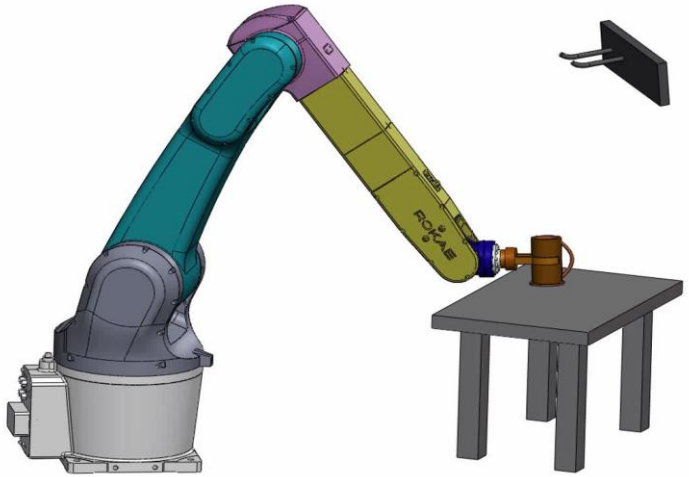
One example of path planning based on RRT

6. Trajectory planning

- **Definition of trajectory**: A time history of position, velocity, and acceleration for each degree of freedom of the manipulator.
- It can be further defined as the state history of $\{T\}$ relative to $\{G\}$

Independent of the type of arm, $\{G\}$ may also vary over time (e.g., conveyor belt)

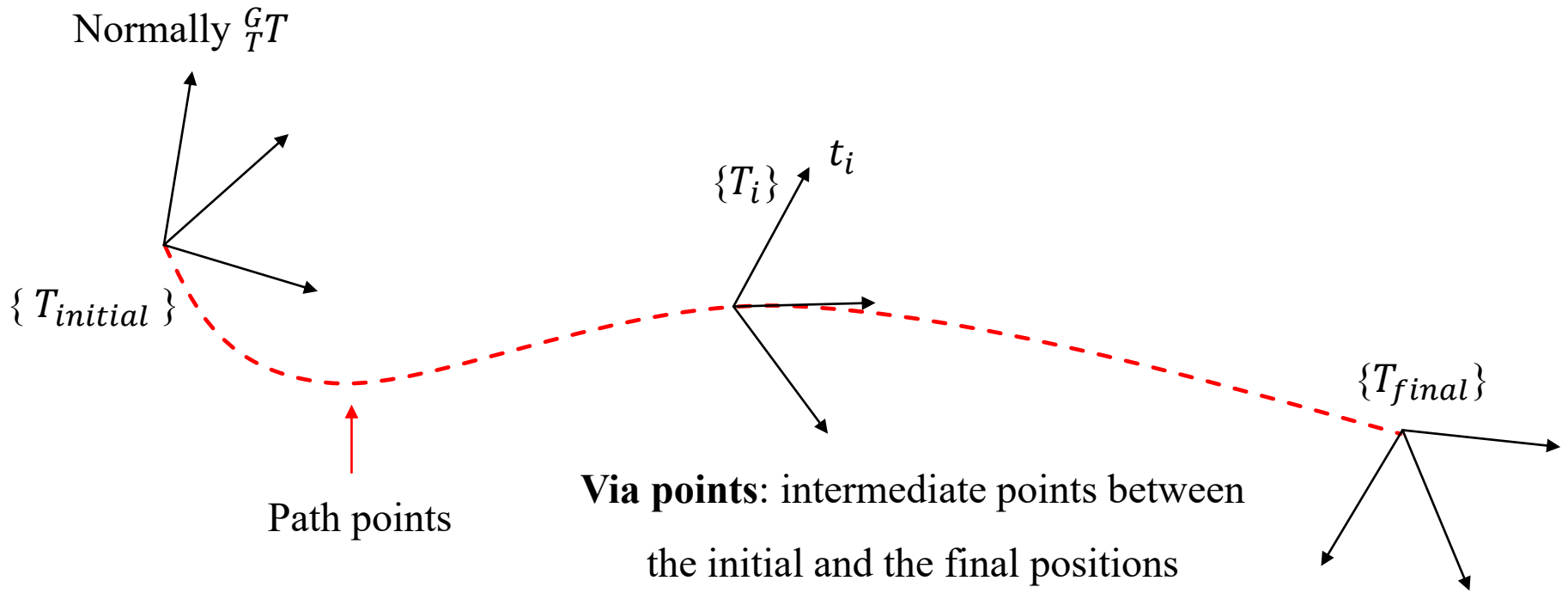




The motion history of the end point of the arm (or object) in an object pick-and-place task is the problem of trajectory planning

6. Trajectory planning

- **Desire trajectory**: smooth path with continuous first derivative



6. Trajectory planning – Joint-space

Steps for trajectory planning in joint space

- **Define initial, via, and final points** of $\{T\}$ respect to $\{G\}$, ${}^{G}_{T}T_{i}$

  (For both translational and rotational states)

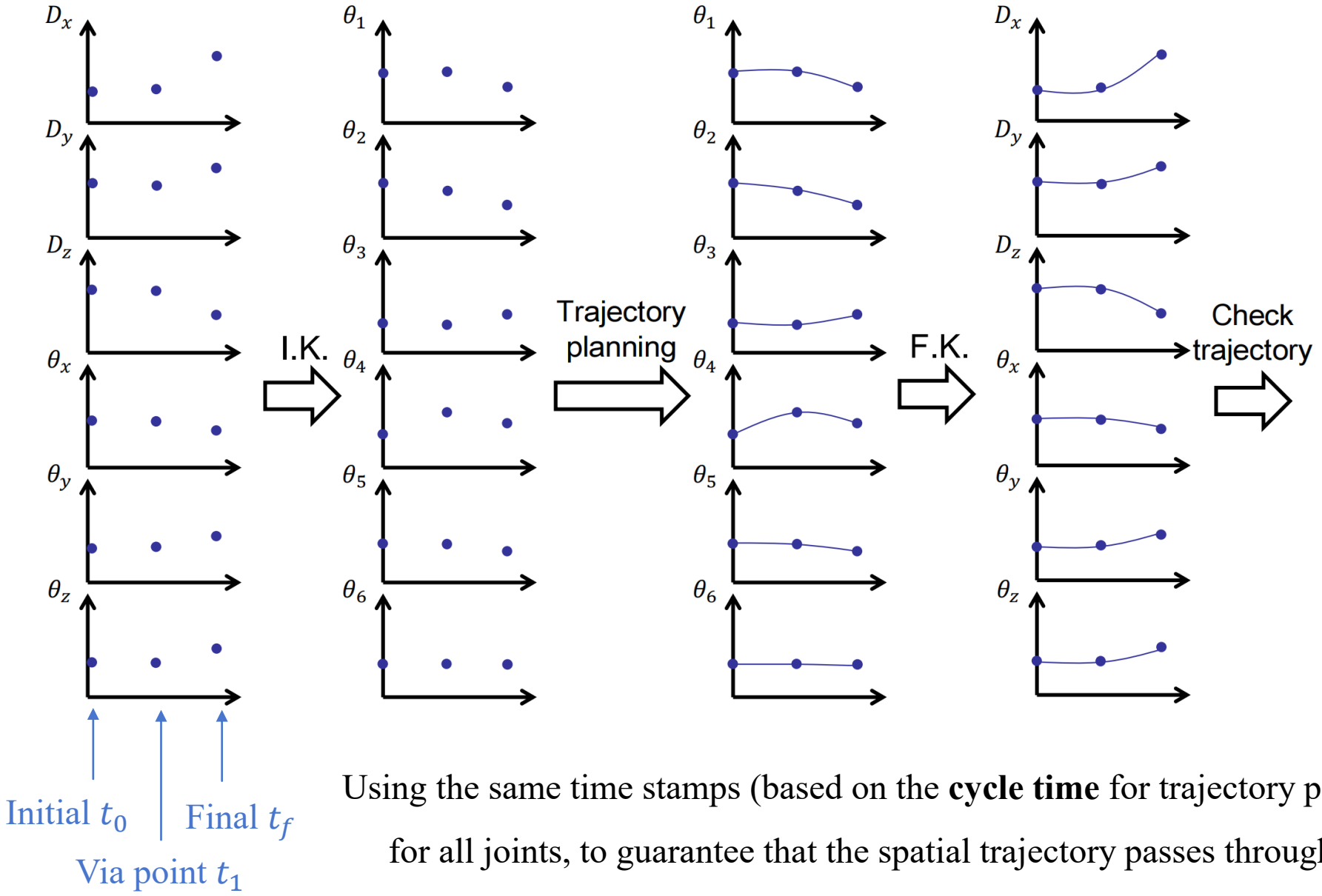
  i=1 initial
  i=2~N-1 via points
  i=N+1 final

Express ${}^{G}_{T}T_{i}$ in a 6-parameter format ${}^{G}X_{T} = \begin{bmatrix} {}^{G}P_{T} \text{ org} \\ ROT\left({}^{G}\hat{K}_{T}, \theta\right) \end{bmatrix}$

Not the rotation matrix

- **Inverse Kinematics**: transforming position and orientation of endpoint of manipulator to joint state: ${}^{G}X_{T} \rightarrow \Theta_{i}$
- **Plan smooth trajectories** for **all joints**.
- **Forward Kinematics**: transforming the joint state to the manipulator in the Cartesian space to check the feasibility of the trajectories under Cartesian-space.

6. Trajectory planning - Joint-space



Using the same time stamps (based on the **cycle time** for trajectory planning) of via points for all joints, to guarantee that the spatial trajectory passes through all the via points

6. Trajectory planning – Cartesian space

Steps for trajectory planning in Cartesian space

- **Define initial, via, and final points** of $\{T\}$ respect to $\{G\}$, ${}^{G}_{T}T_{i}$

  i=1 initial
  i=2~N-1 via points
  i=N+1 final

  (For both translational and rotational states)

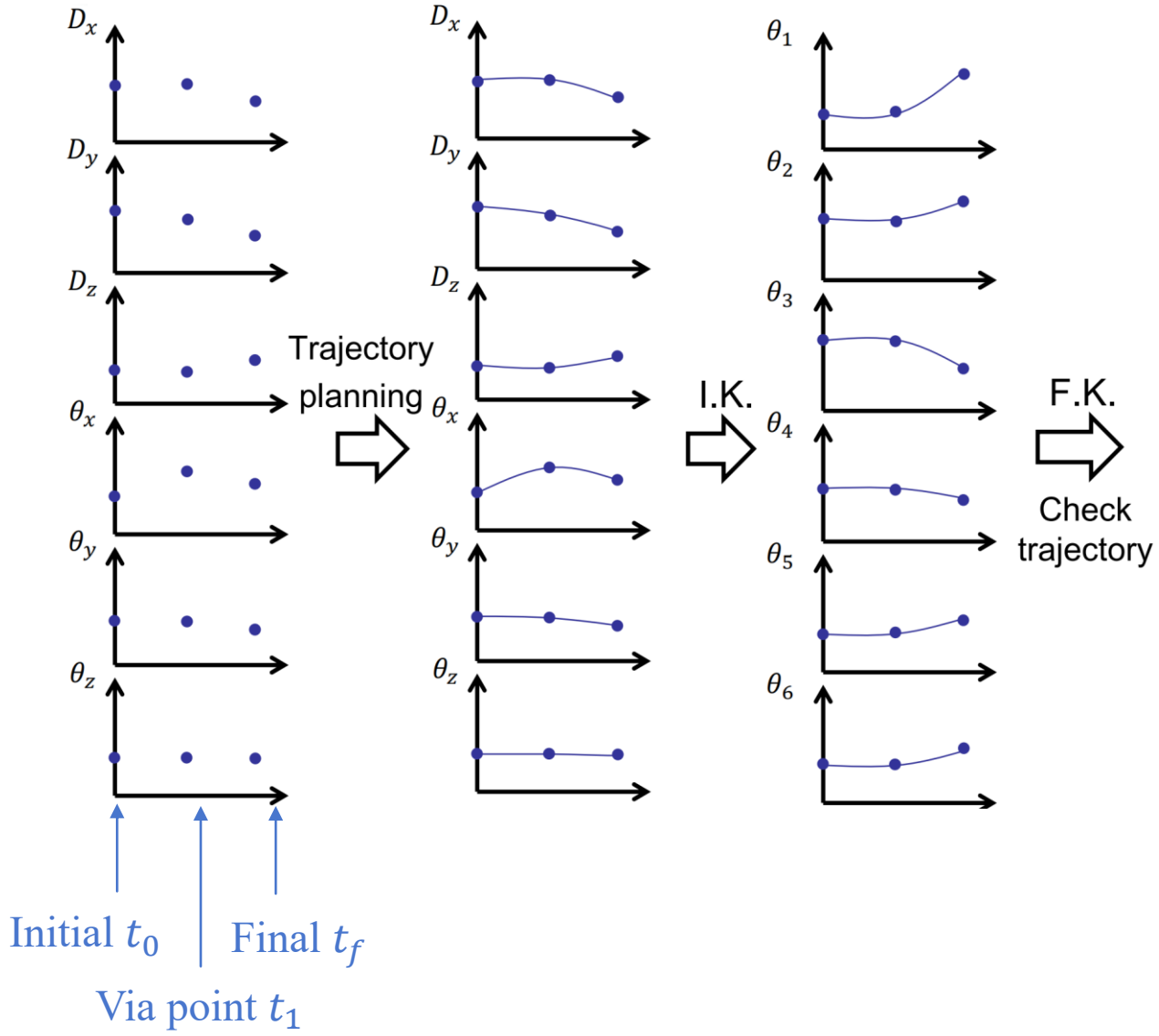
  Express ${}^{G}_{T}T_{i}$ in a 6-parameter format ${}^{G}X_{T} = \begin{bmatrix} {}^{G}P_{T} \text{ org} \\ ROT\left({}^{G}\hat{K}_{T}, \theta\right) \end{bmatrix}$

- **Plan smooth trajectories for position and orientation of a point on the end-effector**
- **Transforms the planed trajectories to the joint state**: ${}^{G}X_{T} \rightarrow \Theta_{i}$
- **Checking the feasibility of joint states for trajectories under the joint-space**

Notes:

- Physically meaningful paths
- **Heavy computation loading**: the IK is performed to calculate the joint angles for each planed trajectory point

6. Trajectory planning – Cartesian space

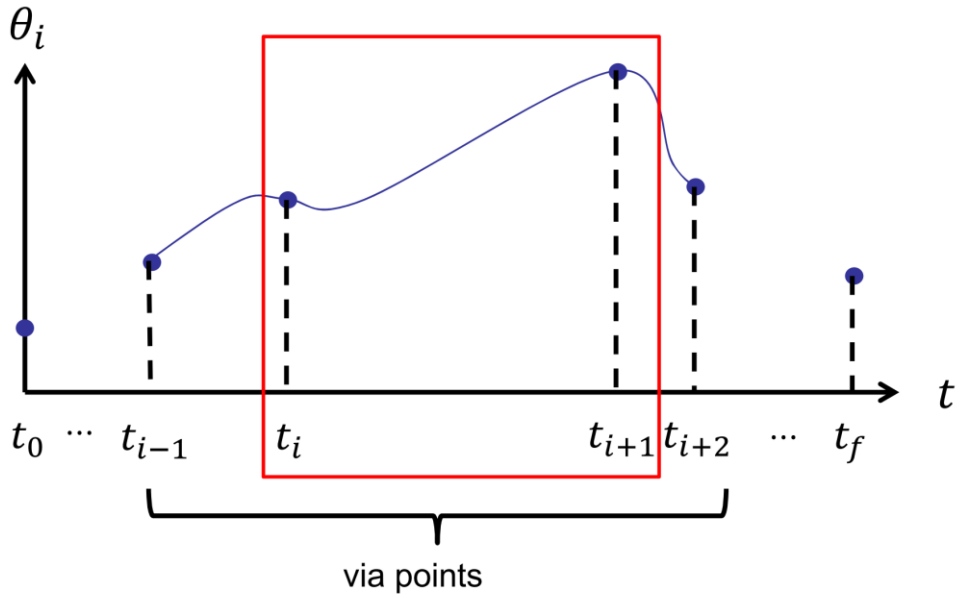


6. Trajectory planning - Cubic Polynomials

Cubic Polynomials

- Trajectory: different time interval $[t_i \quad t_{i+1}]$ use different cubic functions
- Smoothness: boundary conditions defined for each function

  (four conditions for position and velocity: $\theta(t_i), \theta(t_{i+1}), \dot{\theta}(t_i), \dot{\theta}(t_{i+1})$)
- Trajectories are planned in the format of cubic polynomials



1 linear
2 quadratic
3 cubic
4 quartic
5 quintic
6 hexic(sextic)
7 heptic(septic)
8 octic
9 nonic
10 decic

From 1st degree to 10th degree

6. Trajectory planning - Cubic Polynomials

☐ Solve cubic polynomials

- General form

$$\theta(\tilde{t}) = a_0 + a_1\tilde{t} + a_2\tilde{t}^2 + a_3\tilde{t}^3 \qquad \text{4 unknows: } a_j, j = 0\sim3$$

- For each time interval $t \in [t_i \quad t_{i+1}]$

$$\tilde{t} = t - t_i \quad \text{so } \tilde{t}\big|_{t=t_i} = 0 \text{ and } \tilde{t}\big|_{t=t_{i+1}} \equiv \Delta t = t_{i+1} - t_i > 0$$

The Δt of each zone $[t_i, t_{i+1}]$ can be different, depending on the setting of via points

Four boundary conditions

$$\theta\left(\tilde{t}\big|_{t=t_i}\right) = \theta_i = a_0 \qquad ①$$

$$\theta\left(\tilde{t}\big|_{t=t_{i+1}}\right) = \theta_{i+1} = a_0 + a_1\Delta t + a_2\Delta t^2 + a_3\Delta t^3 \qquad ②$$

$$\dot{\theta}\left(\tilde{t}\big|_{t=t_i}\right) = \dot{\theta}_i = a_1 \qquad ③$$

$$\dot{\theta}\left(\tilde{t}\big|_{t=t_{i+1}}\right) = \dot{\theta}_{i+1} = a_1 + 2a_2\Delta t + 3a_3\Delta t^2 \qquad ④$$

☐ Solve the equation ① ② ③ ④

$$a_2 = \frac{3}{\Delta t^2}(\theta_{i+1} - \theta_i) - \frac{2}{\Delta t}\dot{\theta}_i - \frac{1}{\Delta t}\dot{\theta}_{i+1}$$

$$a_3 = -\frac{2}{\Delta t^3}(\theta_{i+1} - \theta_i) + \frac{1}{\Delta t^2}(\dot{\theta}_{i+1} + \dot{\theta}_i)$$

6. Trajectory planning - Cubic Polynomials

Change to matrix representation

$$\begin{bmatrix} \theta_i \\ \theta_{i+1} \\ \dot{\theta}_i \\ \dot{\theta}_{i+1} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & \Delta t & \Delta t^2 & \Delta t^3 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 2\Delta t & 3\Delta t^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

$$\Theta = T_{4\times4}(\Delta t) \cdot A$$

$$\det\left(T_{4\times4}\right) = -\Delta t^4 \neq 0 \text{ as long as } \Delta t$$

Thus, $A = T_{4\times4}^{-1} \cdot \Theta$

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -\frac{3}{\Delta t^2} & \frac{3}{\Delta t^2} & -\frac{2}{\Delta t} & -\frac{1}{\Delta t} \\ \frac{2}{\Delta t^3} & -\frac{2}{\Delta t^2} & \frac{1}{\Delta t^2} & \frac{1}{\Delta t^2} \end{bmatrix} \begin{bmatrix} \theta_i \\ \theta_{i+1} \\ \dot{\theta}_i \\ \dot{\theta}_{i+1} \end{bmatrix}$$

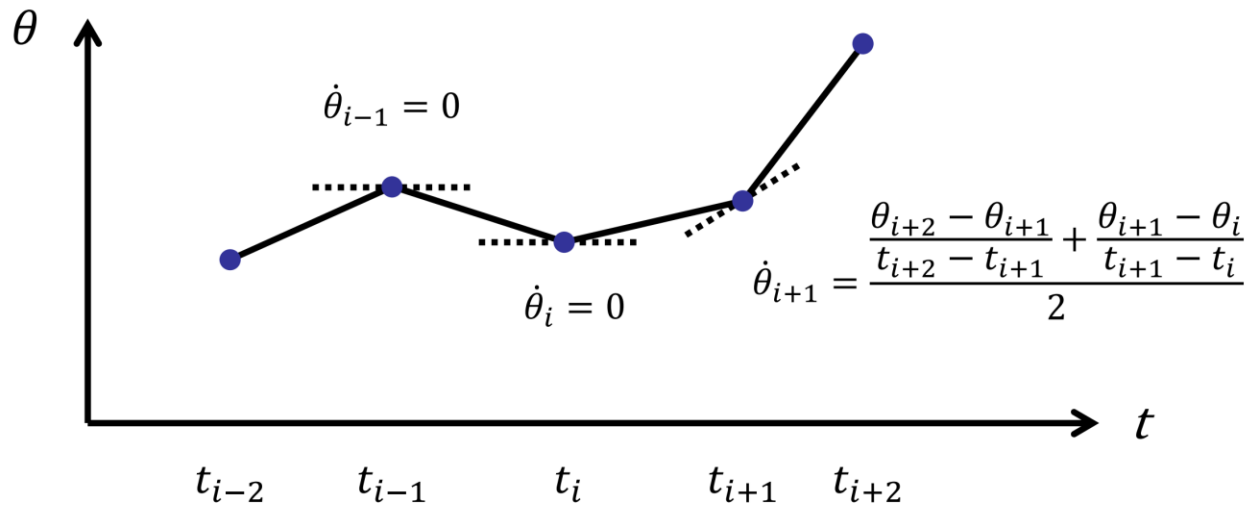
6. Trajectory planning - Cubic Polynomials: Multistage Cubic Polynomial

How to choose velocities $\dot{\theta}_i$ and $\dot{\theta}_{i+1}$?

- **Manually defined velocities** in the via point in either Cartesian space or joint space: complicated, especially around singular points
- **Automatically chooses the velocities** at the via points

  Ex: Choose $\dot{\theta}_i = 0$ if $\dot{\theta}_i$ changes sign before/after t_i

  Choose the average if hold the same sign



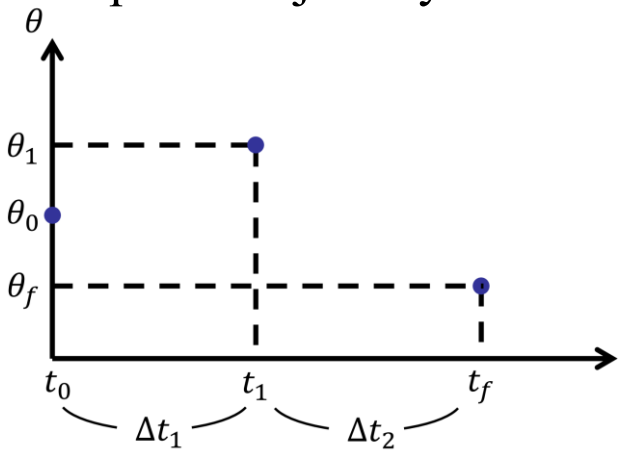
In both ways, the cubic polynomials of different sections can be solved separately

6. Trajectory planning - Cubic Polynomials: Multistage Cubic Polynomial

- Automatically chooses the velocities at the via points in such a way as to make the **acceleration at the via points to be continuous.**

Cubic polynomials from different sections need to be assembled and solved together

- Example: a trajectory with one via point



4 position B.C.s (boundary condition)
2 for each $\theta_j(t)$ j=I , II

$$\begin{bmatrix} \theta_0 = a_{10} \\ \theta_1 = a_{10} + a_{11}\Delta t_1 + a_{12}\Delta t_1^2 + a_{13}\Delta t_1^3 \\ \theta_1 = a_{20} \\ \theta_f = a_{20} + a_{21}\Delta t_2 + a_{22}\Delta t_2^2 + a_{23}\Delta t_2^3 \end{bmatrix}$$

2 velocity B.C.s

$$\begin{bmatrix} \dot{\theta}_0 = 0 = a_{11} \\ \dot{\theta}_f = 0 = a_{21} + 2a_{22}\Delta t_2 + 3a_{23}\Delta t_2^2 \end{bmatrix}$$

not necessary "0"

Via point
Velocity continuity
Acceleration continuity

$$\begin{bmatrix} \dot{\theta}_1 = a_{11} + 2a_{12}\Delta t_1 + 3a_{13}\Delta t_1^2 = a_{21} \\ \ddot{\theta}_1 = 2a_{12} + 6a_{13}\Delta t_1 = 2a_{22} \end{bmatrix}$$

Segments start by translating time from 0

$[t_0 \quad t_1]$ $\tilde{t} = t - t_0$ $[0 \quad \Delta t_1]$

$$\theta_I(\tilde{t}) = a_{10} + a_{11}\tilde{t} + a_{12}\tilde{t}^2 + a_{13}\tilde{t}^3$$

$[t_1 \quad t_f]$ $\tilde{t} = t - t_1$ $[0 \quad \Delta t_2]$

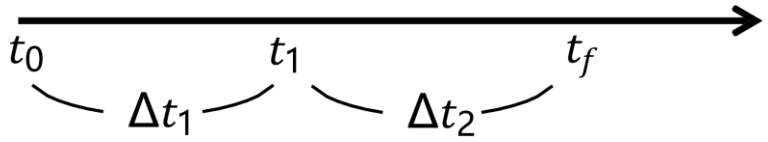
$$\theta_{II}(\tilde{t}) = a_{20} + a_{21}\tilde{t} + a_{22}\tilde{t}^2 + a_{23}\tilde{t}^3$$

6. Trajectory planning - Cubic Polynomials: Multistage Cubic Polynomial

- **Ex: A trajectory with one via point (cont.)**

8 equations, 8 unknowns

Algebraic solution: when $\Delta t_1 = \Delta t_2 = \Delta t$



$$a_{10} = \theta_0$$
$$a_{11} = 0$$
$$a_{12} = \frac{12\theta_1 - 3\theta_f - 9\theta_0}{4\Delta t^2}$$
$$a_{13} = \frac{-8\theta_1 + 3\theta_f + 5\theta_0}{4\Delta t^3}$$

$$a_{20} = \theta_1$$
$$a_{21} = \frac{3\theta_f - 3\theta_0}{4\Delta t}$$
$$a_{22} = \frac{-12\theta_1 + 6\theta_f + 6\theta_0}{4\Delta t^2}$$
$$a_{23} = \frac{8\theta_1 - 5\theta_f - 3\theta_0}{4\Delta t^3}$$

6. Trajectory planning - Cubic Polynomials: Multistage Cubic Polynomial

- **Ex: A trajectory with one via point (cont.)**

Change to matrix representation

$$\Theta_{8\times1} = T_{8\times8} A_{8\times1}$$

$$[0 \quad \Delta t_1] \quad \theta_I(\tilde{t}) = a_{10} + a_{11}\tilde{t} + a_{12}\tilde{t}^2 + a_{13}\tilde{t}^3$$

$$[0 \quad \Delta t_2] \quad \theta_{II}(\tilde{t}) = a_{20} + a_{21}\tilde{t} + a_{22}\tilde{t}^2 + a_{23}\tilde{t}^3$$

$$\begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_1 \\ \theta_f \\ \dot{\theta}_0 \\ \dot{\theta}_f \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & \Delta t_1 & \Delta t_1^2 & \Delta t_1^3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & \Delta t_2 & \Delta t_2^2 & \Delta t_2^3 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 2\Delta t_2 & 3\Delta t_2^2 \\ 0 & 1 & 2\Delta t_1 & 3\Delta t_1^2 & 0 & -1 & 0 & 0 \\ 0 & 0 & 2 & 6\Delta t_1 & 0 & 0 & -2 & 0 \end{bmatrix} \begin{bmatrix} a_{10} \\ a_{11} \\ a_{12} \\ a_{13} \\ a_{20} \\ a_{21} \\ a_{22} \\ a_{23} \end{bmatrix}$$

Assemble two matrixes together

$$A_{8\times1} = T_{8\times8}^{-1} \Theta_{8\times1}$$

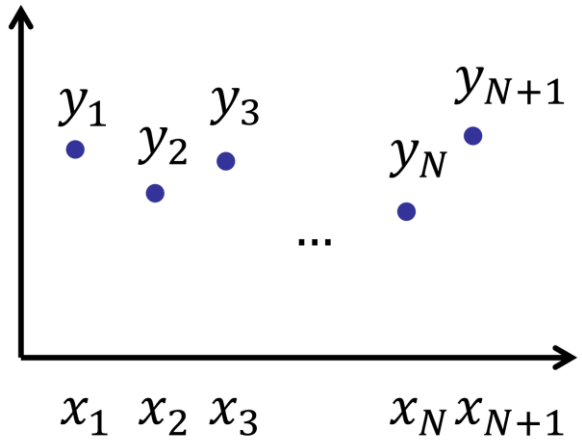
$$\det(T_{8\times8}) = 4\Delta t_1^4 \Delta t_2^3 + 4\Delta t_1^3 \Delta t_2^4$$

$$\neq 0 \text{ as long as } \Delta t_1 \neq 0, \Delta t_2 \neq 0, \Delta t_1 \neq -\Delta t_2)$$

6. Trajectory planning - Cubic Polynomials: General Cubic Polynomial

General cubic spline function

N+1 set points (x_i, y_i) $\begin{cases} 1 & initial \\ N-1 & via \\ 1 & final \end{cases}$



N cubic functions

$$s_j(x) = a_j + b_j x + c_j x^2 + d_j x^3 \qquad x_j \le x \le x_{j+1} \qquad j = 1 \ldots N$$

⇨ Total 4N unknown coefficients

Position conditions at both ends of each $s_j(x)$ ⇨ 2N conditions

Velocity & acceleration continuity conditions at via points ⇨ 2(N-1) conditions

2 MORE CONDITIONS are also needed!

Revisit example: a trajectory with one via point

$$\begin{bmatrix} y_1 = s_1(x_1) \\ y_2 = s_1(x_2) \end{bmatrix} \quad \begin{bmatrix} y_2 = s_2(x_2) \\ y_3 = s_2(x_3) \end{bmatrix}$$

$$\dot{y}_2 = \dot{s}_1(x_2) = \dot{s}_2(x_2)$$

$$\ddot{y}_2 = \ddot{s}_1(x_2) = \ddot{s}_2(x_2)$$

In total 6 conditions

6. Trajectory planning - Cubic Polynomials: General Cubic Polynomial

Choices for the last 2 conditions:

(1) $\ddot{s}_1(x_1) = \ddot{s}_N(x_{N+1}) = 0$ Define acceleration, natural cubic spline

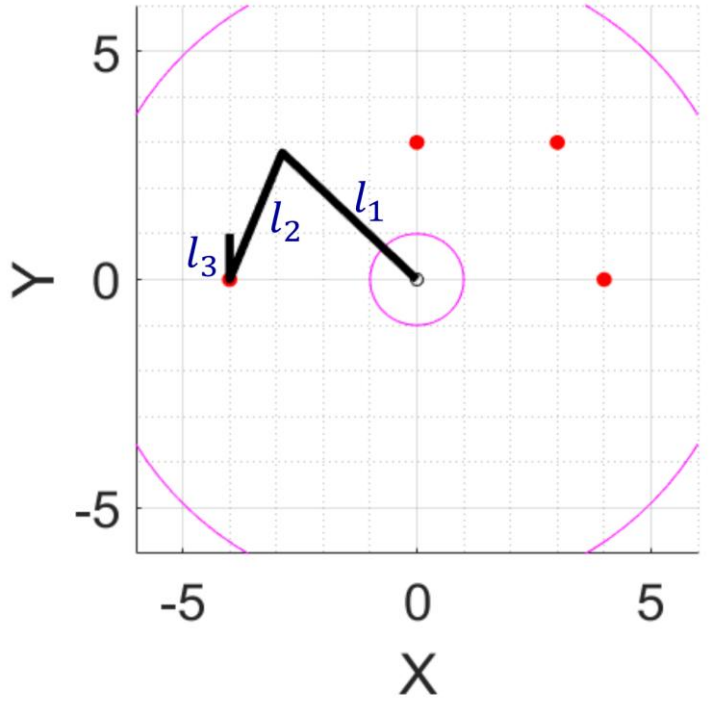
(2) $\dot{s}_1(x_1) = u \quad \dot{s}_N(x_{N+1}) = v$ Define velocity, clamped cubic spline

(3) if $s_1(x_1) = s_N(x_{N+1})$
use $\dot{s}_1(x_1) = \dot{s}_N(x_{N+1})$
$\ddot{s}_1(x_1) = \ddot{s}_N(x_{N+1})$
Continuity of periodic motion, periodic cubic spline

6. Trajectory planning - Cubic Polynomials

- **Ex**: Planar RRR arm length: $l_1 = 4, l_2 = 3, l_3 = 1$. The following table defines the positions of initial, via, via, and final points.

t	x	y	theta
0	-4	0	90
2	0	3	45
4	3	3	30
7	4	0	0



(X,Y) is defined at the end of the second link

θ is the angle of the third link to the X axis

6. Trajectory planning - Cubic Polynomials

☐ **Method 1**: Planning trajectories under Cartesian-space with cubic polynomials

1. Find the coefficients of the respective cubic polynomials of the 3 DOFs (X, Y, θ)

Pass 4 points: 3 cubic polynomials per DOF, total 12 unknowns, i.e., $\Theta_{12\times1} = T_{12\times12} A_{12\times1}$

The function of $(\Delta t_1, \Delta t_2, \Delta t_3)$

$X / Y / \theta$

6 position B.C.s

2 init., final vel.

4 vel. & acl. continuity

$$\begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_1 \\ \theta_2 \\ \theta_2 \\ \theta_f \\ \dot{\theta}_0 \\ \dot{\theta}_f \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & \Delta t_1 & \Delta t_1^2 & \Delta t_1^3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & \Delta t_2 & \Delta t_2^2 & \Delta t_2^3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & \Delta t_3 & \Delta t_3^2 & \Delta t_3^3 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 2\Delta t_3 & 3\Delta t_3^2 \\ 0 & 1 & 2\Delta t_1 & 3\Delta t_1^2 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 6\Delta t_1 & 0 & 0 & -2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 2\Delta t_2 & 3\Delta t_2^2 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 & 6\Delta t_2 & 0 & 0 & -2 & 0 \end{bmatrix} \begin{bmatrix} a_{10} \\ a_{11} \\ a_{12} \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ a_{31} \\ a_{32} \\ a_{33} \end{bmatrix}$$

6. Trajectory planning - Cubic Polynomials

☐ **Method 1**: Planning trajectories under Cartesian-space with cubic polynomials

Substituting the parameters of the three DOFs at each point into MATLAB, it has

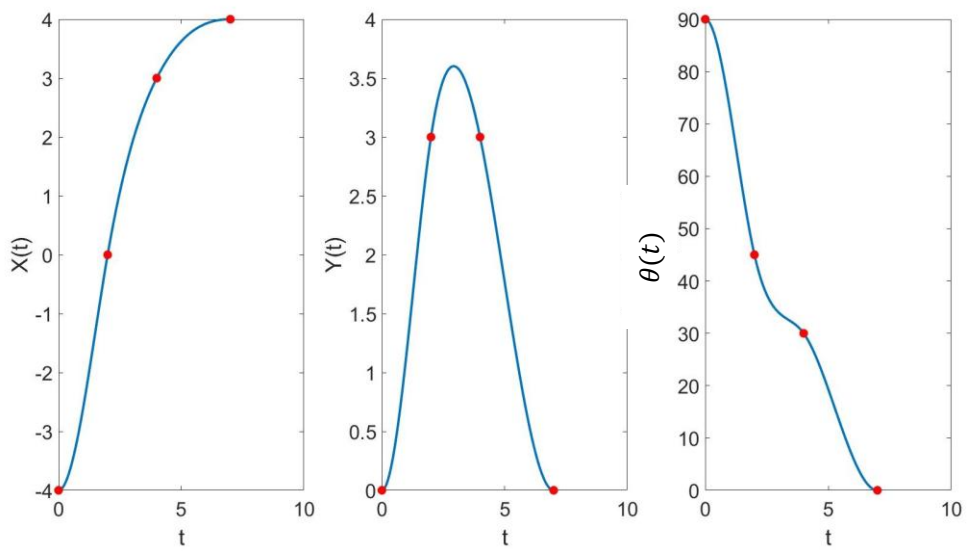
$$\Theta_{12\times1} = T_{12\times12} A_{12\times1}$$

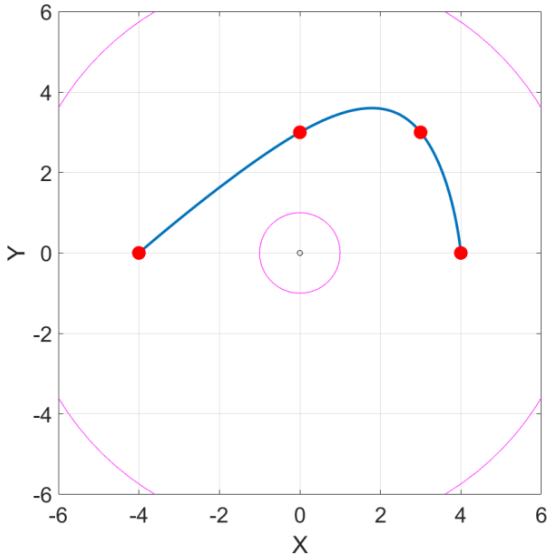
$$\begin{matrix} X & Y & \theta \end{matrix}$$

$$\begin{bmatrix} -4 & 0 & 90 \\ 0 & 3 & 45 \\ 0 & 3 & 45 \\ 3 & 3 & 30 \\ 3 & 3 & 30 \\ 4 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 4 & 8 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 2 & 4 & 8 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 3 & 9 & 27 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 6 & 27 \\ 0 & 1 & 4 & 12 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 12 & 0 & 0 & -2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 4 & 12 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 & 12 & 0 & 0 & -2 & 0 \end{bmatrix} \begin{bmatrix} \\ \\ \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{bmatrix} A_{12\times3}$$

6. Trajectory planning - Cubic Polynomials

2. Plan smooth trajectories for all DOFs

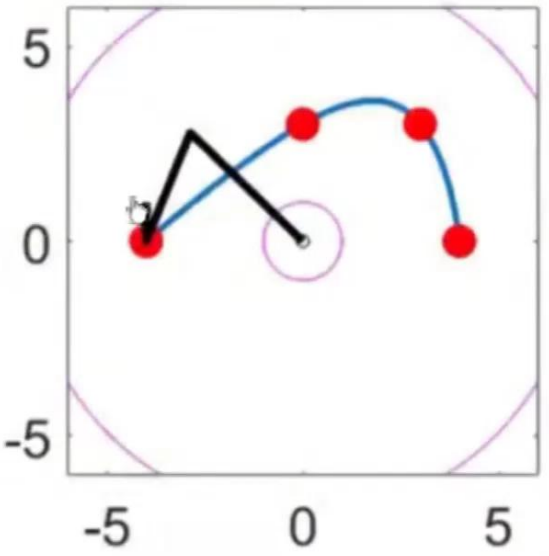




t	x	y	theta
0	-4	0	90
2	0	3	45
4	3	3	30
7	4	0	0

3. I.K., find out the corresponding trajectory of each point.

4. Bring the joints into the arm simulation to confirm that the trajectory of the arm endpoints works as planned.



6. Trajectory planning - Cubic Polynomials

- **Method 2**: Planning trajectories under Joint-space with cubic polynomials

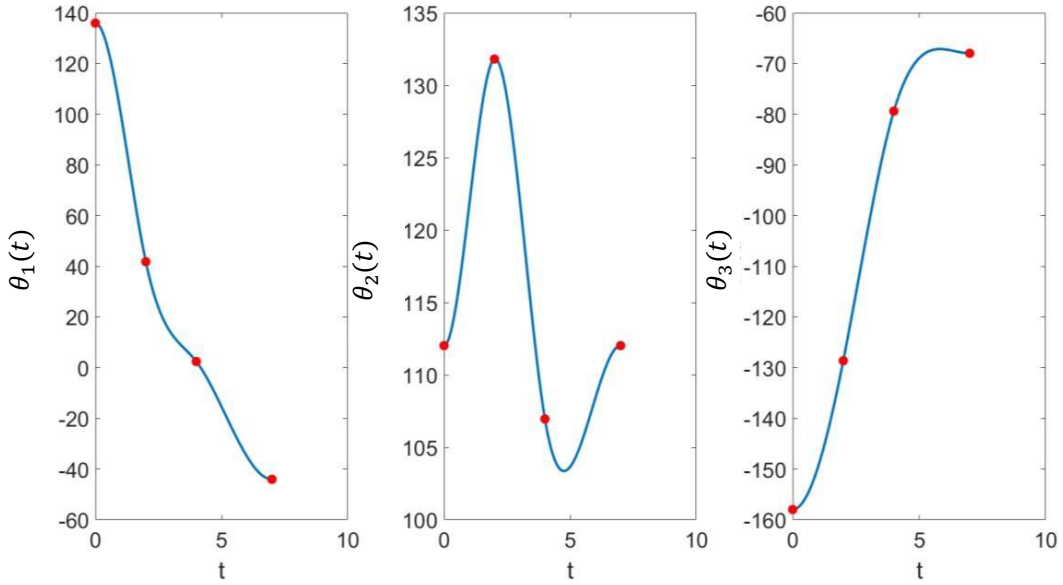
1. **I.K., calculate the joint angles ($\theta_1, \theta_2, \theta_3$) of initial, via and final points**
2. **Calculate the coefficients of each cubic polynomial ($\theta_1, \theta_2, \theta_3$)**

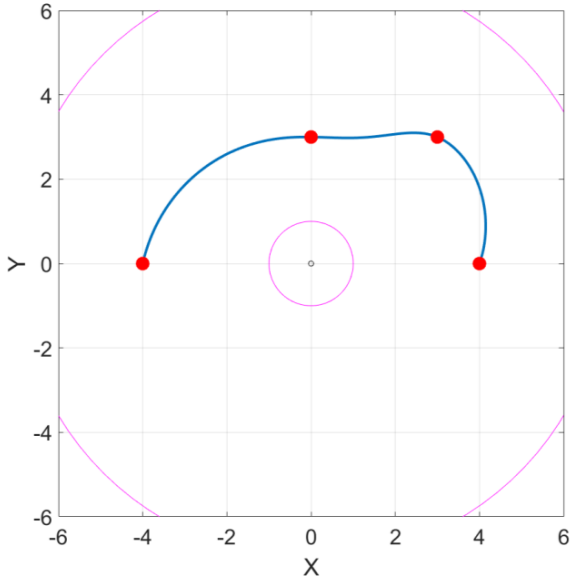
Pass through 4 points: 3 cubic polynomials per joint angle, total 12 unknowns: $\Theta_{12\times3} = T_{12\times12} A_{12\times3}$

$$
\begin{array}{l}
 \\
\text{6 position B.C.s} \\
 \\
 \\
 \\
 \\
 \\
\text{2 init., final vel.} \\
 \\
\text{4 vel. \& acl.} \\
\text{continuity} \\
 \\
 \\
 \\
 \\
\end{array}
\begin{bmatrix}
\theta_1 & \theta_2 & \theta_3 \\
2.3728 & 1.9552 & -2.7572 \\
0.7297 & 2.3005 & -2.2449 \\
0.7297 & 2.3005 & -2.2449 \\
0.0426 & 1.8668 & -1.3858 \\
0.0426 & 1.8668 & -1.3858 \\
-0.7688 & 1.9552 & -1.1864 \\
0 & 0 & 0 \\
0 & 0 & 0 \\
0 & 0 & 0 \\
0 & 0 & 0 \\
0 & 0 & 0 \\
0 & 0 & 0
\end{bmatrix}
=
\begin{bmatrix}
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
1 & 2 & 4 & 8 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 2 & 4 & 8 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 3 & 9 & 27 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 6 & 27 \\
0 & 1 & 4 & 12 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 2 & 12 & 0 & 0 & -2 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 4 & 12 & 0 & -1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 2 & 12 & 0 & 0 & -2 & 0
\end{bmatrix}
\begin{bmatrix}
 \\
 \\
 \\
 \\
 \\
 \\
 \\
 \\
 \\
 \\
 \\
 \\
\end{bmatrix}
A_{12\times3}
$$

6. Trajectory planning - Cubic Polynomials

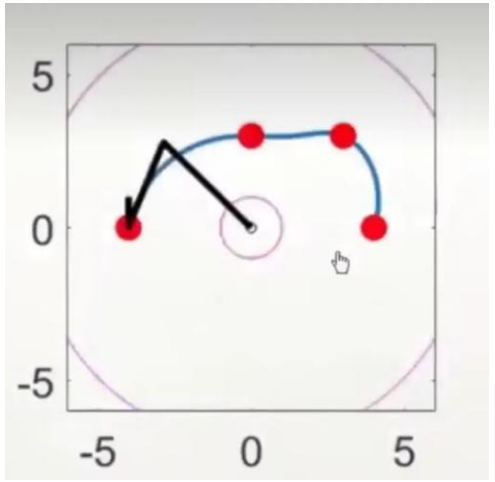
3. Planning smooth trajectories for all joints

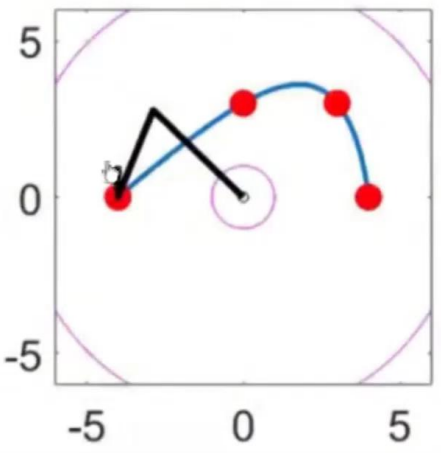




t	x	y	theta
0	-4	0	90
2	0	3	45
4	3	3	30
7	4	0	0

4. Bring the joints into the arm to simulate movement and confirm that the arm end point trajectory works as planned.





Joint-space vs. Cartesian-space

6. Trajectory planning - High-order Polynomials

☐ If position, velocity, and acceleration all have conditions

⇨ Quintic polynomial $\theta(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5 = \sum_{i=0}^{5} a_i t^i$

6 conditions, quintic polynomial with 6 unknowns

$$\begin{cases} \theta_0 = a_0 \\ \theta_f = a_0 + a_1 \Delta t + a_2 \Delta t^2 + a_3 \Delta t^3 + a_4 \Delta t^4 + a_5 \Delta t^5 \\ \dot{\theta}_0 = a_1 \\ \dot{\theta}_f = a_1 + 2a_2 \Delta t + 3a_3 \Delta t^2 + 4a_4 \Delta t^3 + 5a_5 \Delta t^4 \\ \ddot{\theta}_0 = 2a_2 \\ \ddot{\theta}_f = 2a_2 + 6a_3 \Delta t + 12a_4 \Delta t^2 + 20a_5 \Delta t^3 \end{cases}$$

⇨

$$a_0 = \theta_0 \qquad a_3 = \frac{20(\theta_f - \theta_0) - (8\dot{\theta}_f + 12\dot{\theta}_0)\Delta t - (3\ddot{\theta}_0 - \ddot{\theta}_f)\Delta t^2}{2\Delta t^3}$$

$$a_1 = \dot{\theta}_0 \qquad a_4 = \frac{30(\theta_0 - \theta_f) + (14\dot{\theta}_f + 16\dot{\theta}_0)\Delta t - (3\ddot{\theta}_0 - 2\ddot{\theta}_f)\Delta t^2}{2\Delta t^4}$$

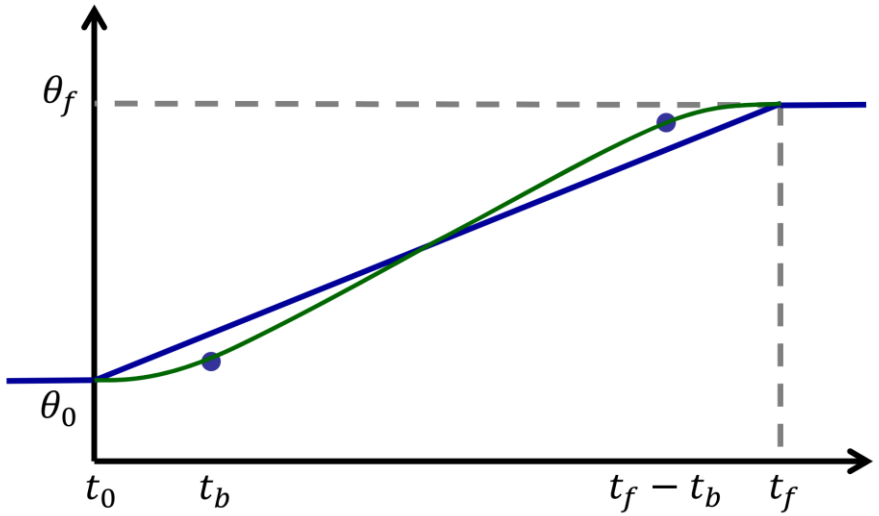
$$a_2 = \frac{1}{2}\ddot{\theta}_0 \qquad a_5 = \frac{12(\theta_f - \theta_0) - (6\dot{\theta}_f + 6\dot{\theta}_0)\Delta t - (\ddot{\theta}_0 - \ddot{\theta}_f)\Delta t^2}{2\Delta t^5}$$

6. Trajectory planning - Linear Function with Parabolic Blends

- Straight-line trajectories are required for many types of tasks.
- If the trajectory contains more than one straight-line segment trajectory, the velocity is not continuous at the turning points between the segments.
- Modify the **ends of the straight-line segments to a quadratic equation** to make the velocity trajectory smooth.

⇩

Linear Function with Parabolic Blends



$(assume\ \ \dot{\theta}_0 = \dot{\theta}_f = 0)$

6. Linear Function with Parabolic Blends - Two Consecutive Line Segments

☐ Formulation

- Linear section (polynomial in one order): constant velocity

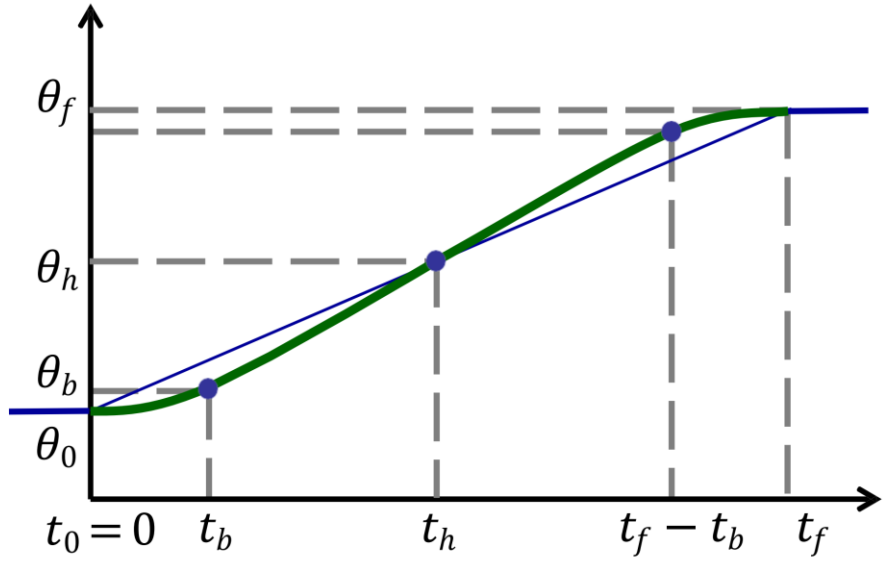
$$\dot{\theta} = \frac{\theta_h - \theta_b}{t_h - t_b} = \dot{\theta}_{t_b} \qquad \text{...①}$$

- First quadratic section (quadratic polynomial): constant acceleration

$$\theta(t) = \theta_0 + \dot{\theta} t + \frac{1}{2}\ddot{\theta} t^2$$

$$\dot{\theta}(t) = \dot{\theta} + \ddot{\theta} t$$

$$\dot{\theta}(t_b) = \ddot{\theta} t_b \qquad \text{...②}$$



$$t_h = \frac{1}{2} t_f \qquad \theta_h = \frac{\theta_f + \theta_0}{2}$$

assume $\dot{\theta}_0 = \dot{\theta}_f = 0$

6. Linear Function with Parabolic Blends - Two Consecutive Line Segments

- **Junction velocity needs to be continuous**

② = ①

$$\ddot{\theta} t_b = \dot{\theta}_{t_b} = \frac{\theta_h - \theta_b}{t_h - t_b} = \frac{\frac{\theta_f + \theta_0}{2} - \left(\theta_0 + \frac{1}{2}\ddot{\theta} t_b^2\right)}{\frac{t_f}{2} - t_b} = \frac{\theta_f - \theta_0 - \ddot{\theta} t_b^2}{t_f - 2t_b}$$

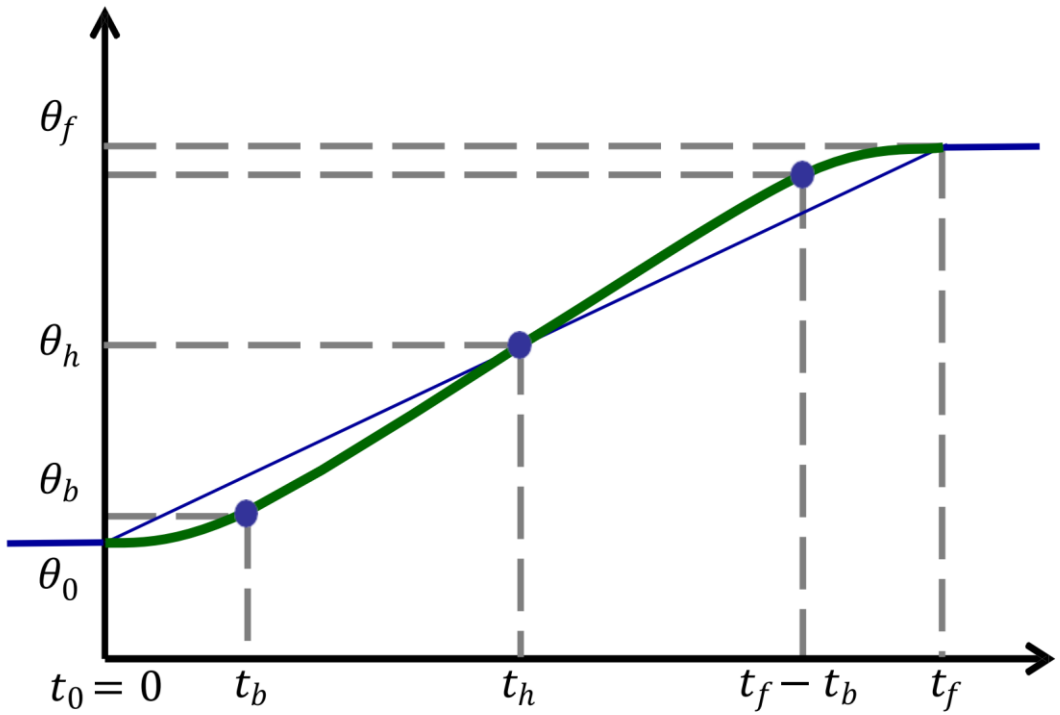
$$\ddot{\theta} t_b^2 - \ddot{\theta} t_f t_b + \left(\theta_f - \theta_0\right) = 0$$

$$\Rightarrow \quad t_b = \frac{\ddot{\theta} t_f - \sqrt{\ddot{\theta}^2 t_f^2 - 4\ddot{\theta}\left(\theta_f - \theta_0\right)}}{2\ddot{\theta}}$$

The discriminant requires a positive number or 0 to obtain t_b as a real number

$$\ddot{\theta} \geq \frac{4\left(\theta_f - \theta_0\right)}{t_f^2}$$

$$\ddot{\theta}_{\min} = \frac{4\left(\theta_f - \theta_0\right)}{t_f^2}$$



6. Linear Function with Parabolic Blends - Two Consecutive Line Segments

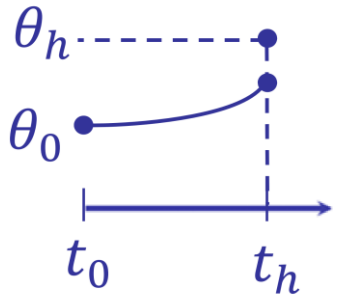
- **Discussion of acceleration $\ddot{\theta}$ state**
  - If $\ddot{\theta} = \ddot{\theta}_{min}$ $\quad t_b = \frac{t_f}{2} = t_h$ No linear segments, two parabolic segments are connected.

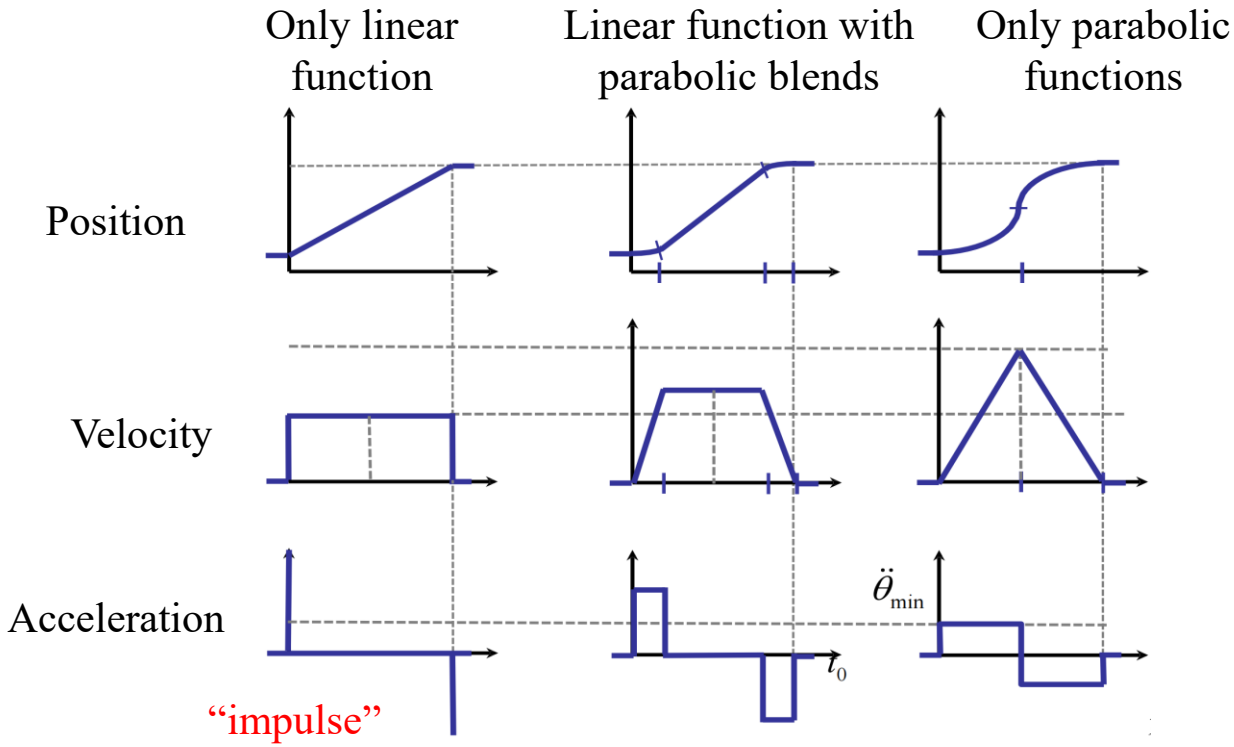
$$\text{at } t_b : \dot{\theta}(t_b) = \ddot{\theta} t_b = \frac{4(\theta_f - \theta_0)}{t_f^2}\frac{t_f}{2} = 2\frac{\theta_f - \theta_0}{t_f}$$

Twice the velocity comparing to "linear only" situation

  - If $\ddot{\theta} < \ddot{\theta}_{min}$

Acceleration is not enough at $t_b = t_h, \theta < \theta_h$



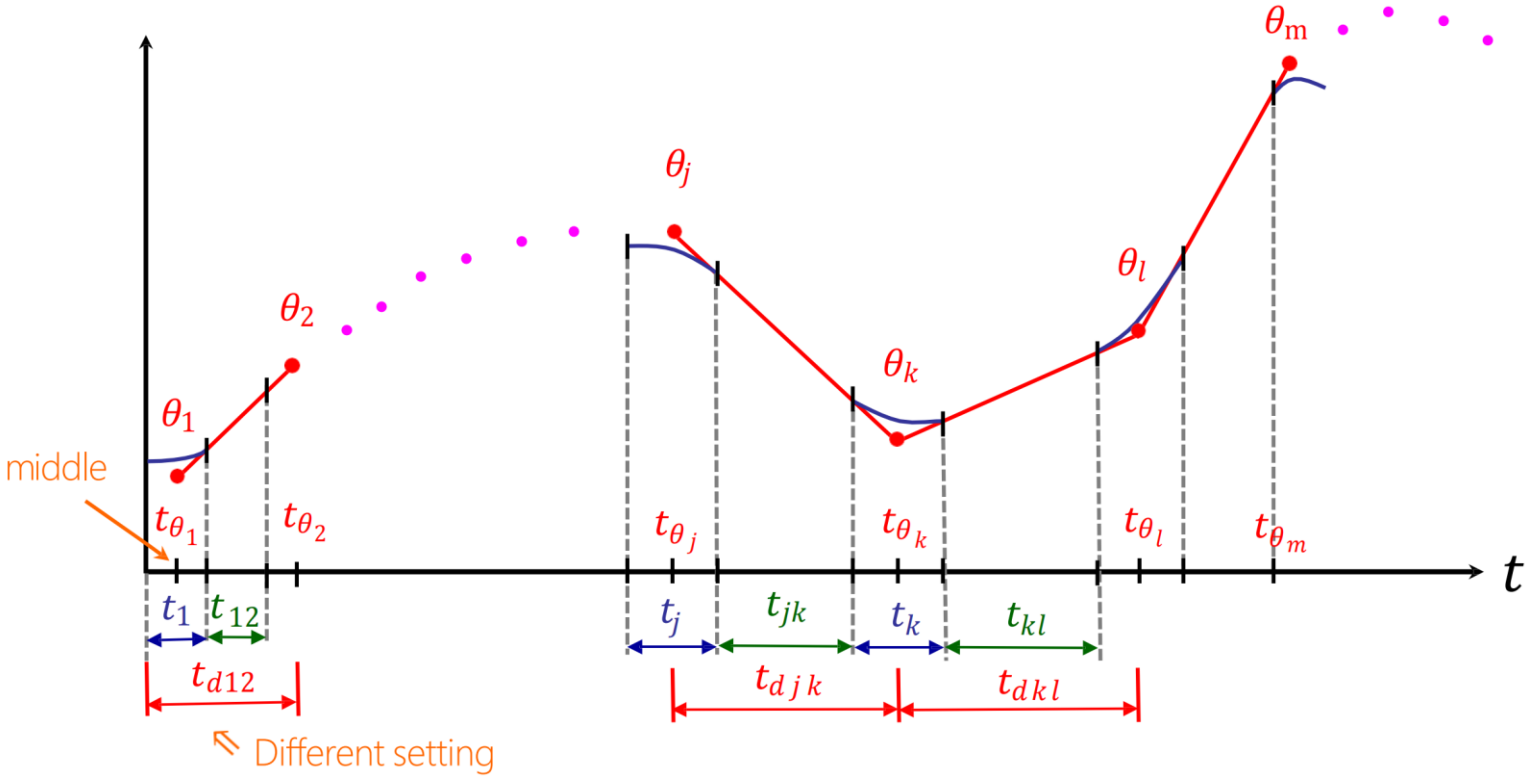


Comparison of different cases

6. Linear Function with Parabolic Blends - Multiple Consecutive Line Segments

☐ General case: a path with *n* via points

Each segment $[\theta_i \quad \theta_{i+1}]$ is equivalent to a single linear segment $[\theta_0 \quad \theta_f]$ as in the previous example, but the velocities of the segments before and after this segment are non zero.



6. Linear Function with Parabolic Blends - Multiple Consecutive Line Segments

☐ For any line segment $[\theta_i \quad \theta_{i+1}]$

- **Linear (polynomial in one order)**

$$\dot{\theta}_{jk} = \frac{\theta_k - \theta_j}{t_{djk}} \qquad \dot{\theta}_{kl} = \frac{\theta_l - \theta_k}{t_{dkl}}$$

- **Quadratic section (quadratic polynomial)**

Method 1: Set acceleration to solve for time

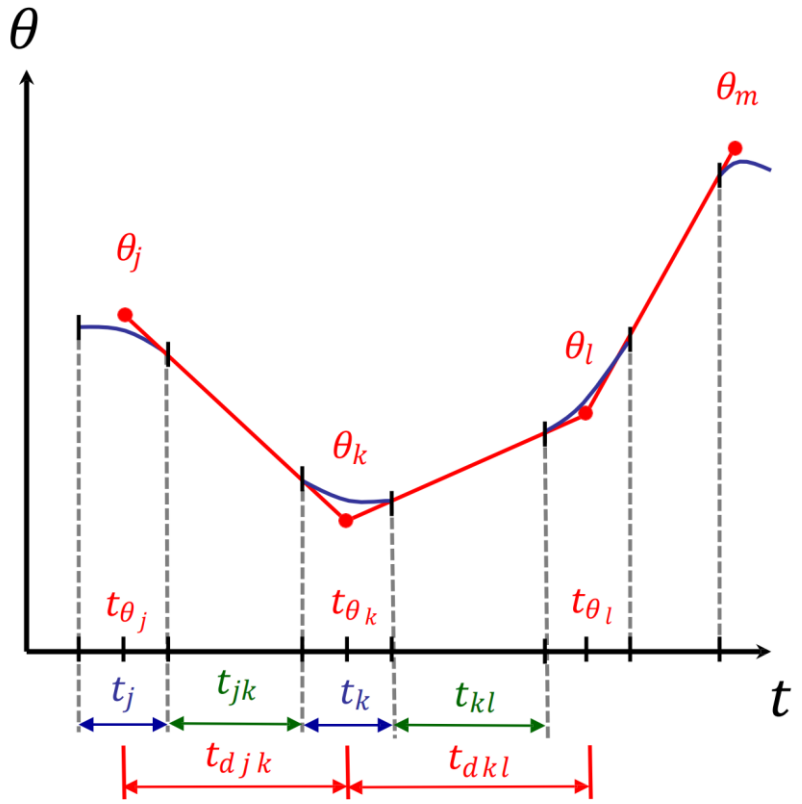
$$\ddot{\theta}_k = \mathrm{sgn}\left(\dot{\theta}_{kl} - \dot{\theta}_{jk}\right)\left|\ddot{\theta}_k\right|$$

$$t_k = \frac{\dot{\theta}_{kl} - \dot{\theta}_{jk}}{\ddot{\theta}_k}$$ Set acceleration

Method 2: Set time to solve for acceleration

$$\ddot{\theta}_k = \frac{\dot{\theta}_{kl} - \dot{\theta}_{jk}}{t_k}$$ Set time

$$t_{jk} = t_{djk} - \frac{1}{2}t_j - \frac{1}{2}t_k$$



6. Linear Function with Parabolic Blends - Multiple Consecutive Line Segments

☐ The first segment

- θ_1 can be regarded as the whole trajectory start point θ_0 shifted backward in time (half of the time t_1 required for the parabolic curve segment) to insert the parabolic curve segment so that the velocity can be continuous from the start point.

Method 1: Set acceleration to solve for time

$$\ddot{\theta}_1 = \text{sgn}(\theta_2 - \theta_1)\left|\ddot{\theta}_1\right| \quad \text{Set acceleration}$$

$$\dot{\theta}_{12} = \frac{\theta_2 - \theta_1}{t_{d12} - \frac{1}{2}t_1} = \ddot{\theta}_1 t_1$$

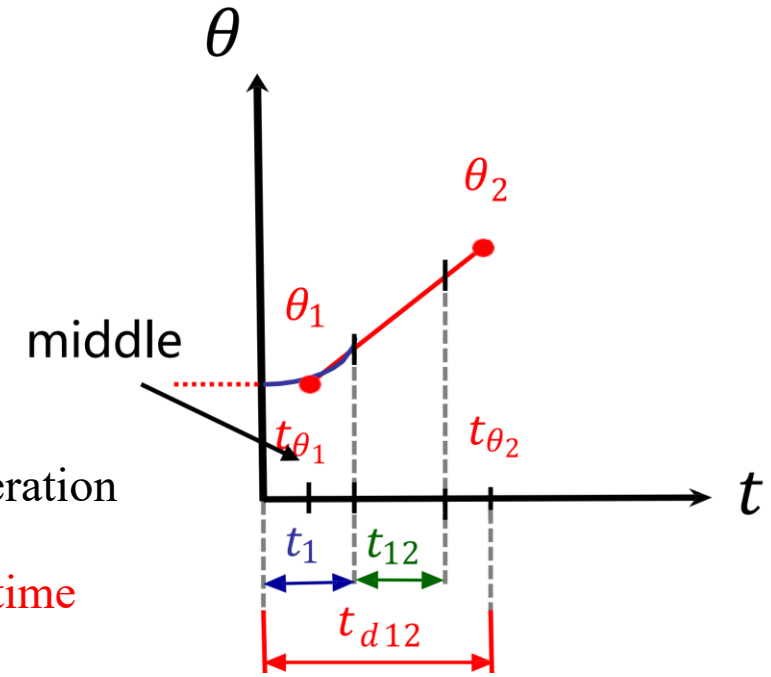
$$\frac{1}{2}\ddot{\theta}_1 t_1^2 - t_{d12}\ddot{\theta}_1 t_1 + (\theta_2 - \theta_1) = 0$$

$$t_1 = t_{d12} - \sqrt{t_{d12}^2 - \frac{2(\theta_2 - \theta_1)}{\ddot{\theta}_1}}$$

Method 2: Set time to solve for acceleration

$$\ddot{\theta}_1 = \frac{\theta_2 - \theta_1}{\left(t_{d12} - \frac{1}{2}t_1\right)}\frac{1}{t_1} \quad \text{Set time}$$

$$t_{12} = t_{d12} - t_1 - \frac{1}{2}t_2$$



6. Linear Function with Parabolic Blends - Multiple Consecutive Line Segments

☐ The last segment

- θ_n can be regarded as the endpoint of the whole trajectory θ_f shifted forward in time (half of the time t_n required for the parabolic curve segment) to insert the parabolic curve segment so that the velocity is continuous

Method 1: Set acceleration to solve for time

$$\ddot{\theta}_n = \text{sgn}\left(\theta_n - \theta_{n-1}\right)\left|\ddot{\theta}_n\right| \quad \text{Set acceleration}$$

$$\dot{\theta}_{(n-1)n} = \frac{\theta_n - \theta_{n-1}}{t_{d(n-1)n} - \frac{1}{2}t_n} = \ddot{\theta}_n\left(-t_n\right)$$

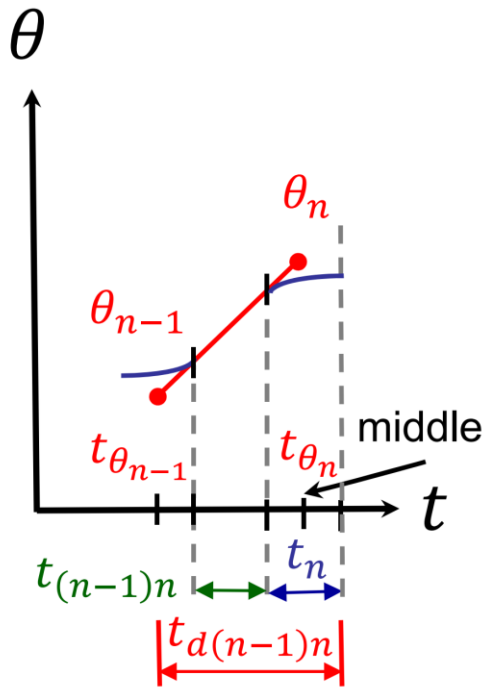
$$\frac{1}{2}\ddot{\theta}_n t_n^2 - t_{d(n-1)n}\ddot{\theta}_n t_n + \left(\theta_n - \theta_{n-1}\right) = 0$$

$$t_n = t_{d(n-1)n} - \sqrt{t_{d(n-1)n}^2 - \frac{2\left(\theta_n - \theta_{n-1}\right)}{\ddot{\theta}_n}}$$

Method 2: Set time to solve for acceleration

$$\ddot{\theta}_n = \frac{\theta_n - \theta_{n-1}}{\left(t_{d(n-1)n} - \frac{1}{2}t_n\right)}\frac{1}{-t_n} \quad \text{Set time}$$

$$t_{(n-1)n} = t_{d(n-1)n} - t_n - \frac{1}{2}t_{n-1}$$



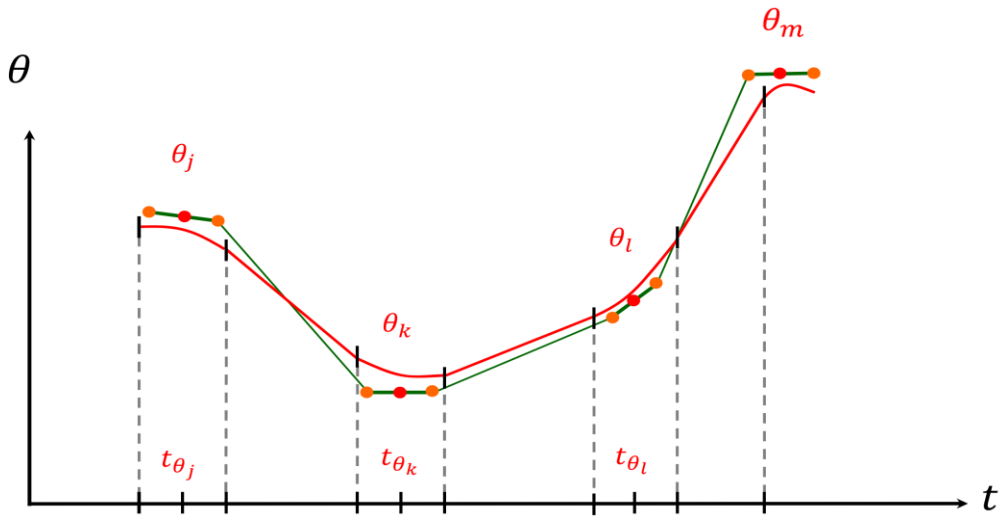
6. Linear Function with Parabolic Blends - Discussions

☐ The acceleration $\ddot{\theta}$ achievable in a real system depends on many factors

- Capabilities of motor
- Posture of manipulator: the torque required (e.g. due to gravity and working load) by each axis is different for different postures of manipulator
- Dynamic state of manipulator: each axis needs to carry different inertia forces under different posture of manipulator

☐ The planned trajectory did not pass through via points

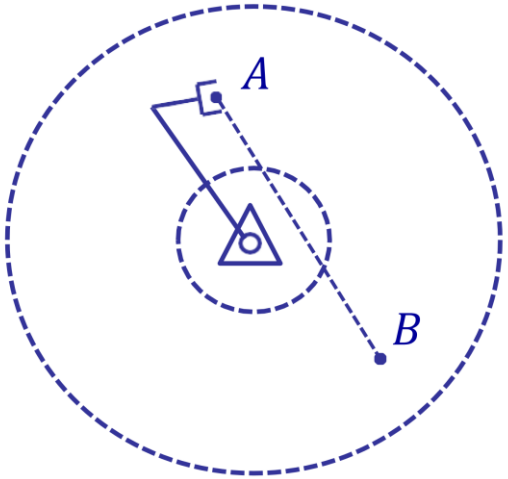
- Only trajectories with acceleration $\rightarrow \infty$ can pass the via points
- If passing the via points are required ⇒ Create pseudo via points so that the original via points fall on the linear segment and they will be passed.

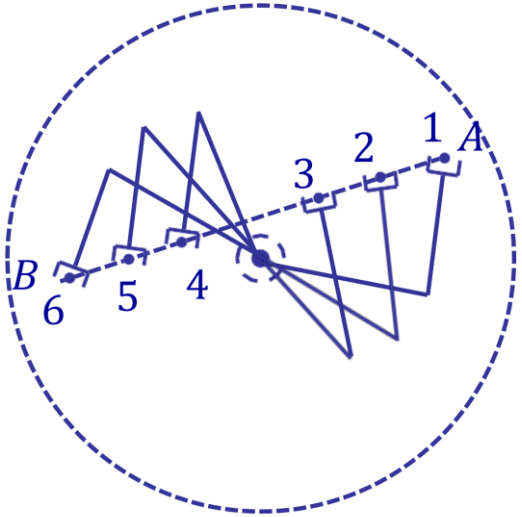


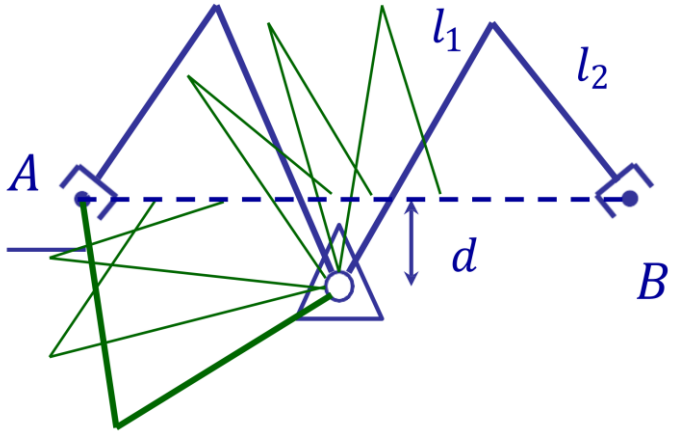
6. Linear Function with Parabolic Blends

☐ Geometric constraints on trajectories in Cartesian space

- Some intermediate segments are unreachable (i.e., outside the work space)
- Trajectory needs high acceleration and deceleration (i.e., close to singular configuration)
- Continuous trajectories cannot be generated for specific start and end postures, e.g., as follows, unless $l_1 - l_2 = d$







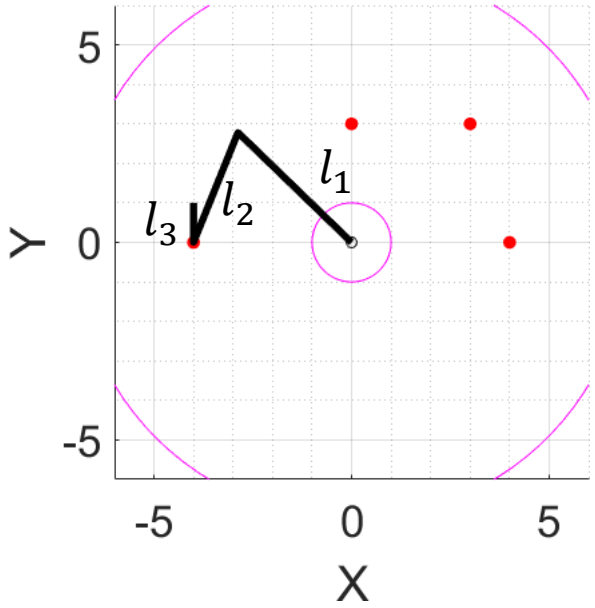
6. Linear Function with Parabolic Blends - Example: RRR Manipulator

- **Ex**: Planar RRR arm length: $l_1 = 4, l_2 = 3, l_3 = 1$, the following table defines the positions of initial, via, via, and final points
- **Method**: Planning trajectories in Cartesian space with linear function with parabolic blends

i	t_i	x_i	y_i	θ_i
0	0	-4	0	90
1	2	0	3	45
2	4	3	3	30
3	7	4	0	0

(X,Y) is defined at the end of the second link

and θ is the angle of the third link to the X axis



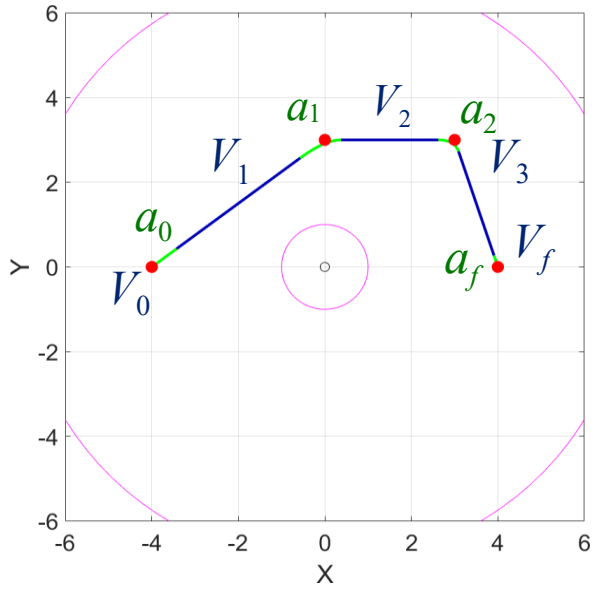
6. Linear Function with Parabolic Blends - Example: RRR Manipulator

1. Find the velocity and acceleration of each DOF (X, Y, θ) in each segment.

(Each Parabolic function interval is 0.5 seconds)

i	t_i	x_i	y_i	θ_i
0	0	-4	0	90
1	2	0	3	45
2	4	3	3	30
3	7	4	0	0

	X	Y	θ(deg/s)
V_0	0	0	0
V_1	2.29	1.71	-25.71
V_2	1.5	0	-7.5
V_3	0.36	-1.09	-10.9
V_f	0	0	0



	X	Y	θ(deg/s²)
a_0	4.57	3.43	-51.4
a_1	-1.57	-3.43	36.4
a_2	-2.27	-2.18	-6.82
a_f	-0.72	2.18	21.8

Middle line segment

$$V_2 = \frac{DOF_2 - DOF_1}{4-2}$$

The first and last line segments

$$V_1 = \frac{DOF_1 - DOF_0}{2-0-\frac{0.5}{2}}$$

$$V_3 = \frac{DOF_3 - DOF_2}{7-4-\frac{0.5}{2}}$$

$$a_0 = \frac{V_1 - V_0}{0.5}$$

$$a_1 = \frac{V_2 - V_1}{0.5}$$

$$a_2 = \frac{V_3 - V_2}{0.5}$$

$$a_3 = \frac{V_f - V_3}{0.5}$$

6. Linear Function with Parabolic Blends - Example: RRR Manipulator

2. Build the equation of each DOF (*X*, *Y*, θ) in each segment. (Linear/Parabolic total 7 segments)

Take X for example,

$$X_{eq1}(t) = X_0 + V_0\Delta t + \frac{1}{2}a_0\Delta t^2 \quad = -4 + 0(t-0) + \frac{1}{2}4.57(t-0)^2 \qquad t \in [0, 0.5]$$

$$X_{eq2}(t) = X_0 + V_1\Delta t \quad = -4 + 2.29(t-0.25) \qquad t \in [0.5, 1.75]$$

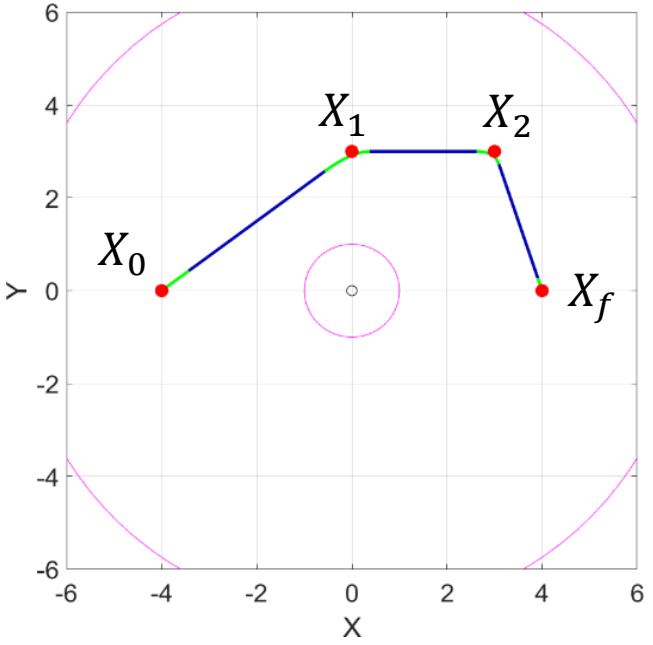
$$X_{eq3}(t) = X_0 + V_1\Delta t_1 + \frac{1}{2}a_1\Delta t_2^2 \quad = -4 + 2.29(t-0.25) + \frac{1}{2}(-1.57)(t-1.75)^2 \qquad t \in [1.75, 2.25]$$

$$X_{eq4}(t) = X_1 + V_2\Delta t \quad = 0 + 1.5(t-2) \qquad t \in [2.25, 3.75]$$

$$X_{eq5}(t) = X_1 + V_2\Delta t_1 + \frac{1}{2}a_2\Delta t_2^2 \quad = 0 + 1.5(t-2) + \frac{1}{2}(-2.27)(t-3.75)^2 \qquad t \in [3.75, 4.25]$$

$$X_{eq6}(t) = X_2 + V_3\Delta t \quad = 3 + 0.36(t-4) \qquad t \in [4.25, 6.5]$$

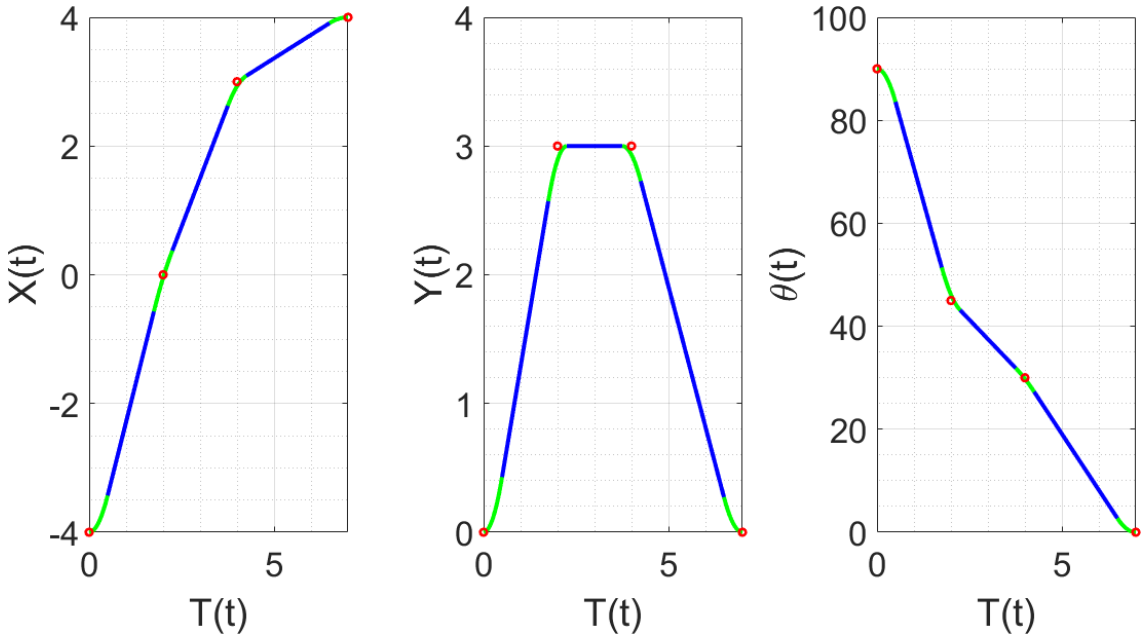
$$X_{eq7}(t) = X_2 + V_3\Delta t_1 + \frac{1}{2}a_3\Delta t_2^2 \quad = 3 + 0.36(t-4) + \frac{1}{2}(-0.73)(t-6.5)^2 \qquad t \in [6, 5, 7]$$

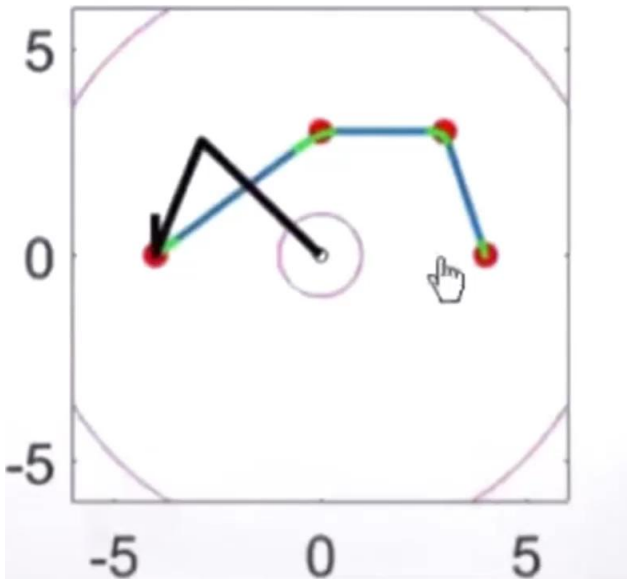


6. Linear Function with Parabolic Blends - Example: RRR Manipulator

3. Plot the planning trajectory for each DOFs

4. Calculate the angles of the 3 rotational axis with IK; substitute the arm parameters into FK, and plot the spatial motion trajectory of the arm to confirm the correctness of the trajectory planning





Summary of the Chapter

- The concepts of configuration space and how it represents the states of a robot.
- The methods for $Q = R^2$, graph-based method, artificial potential field and sampling-based methods to plan paths.
- Use cubic polynomials to plan continuous motion trajectories of position, velocity, and acceleration.
- Use linear functions with parabolic blends to plan a motion trajectory that can move in a straight line at a constant velocity.

Path and Trajectory Planning

- Configuration space
- Path planning for $Q = R^2$
- Artificial potential field
- Sampling-based methods
- Trajectory planning